Codex for Claude Code Users: A Practical Guide to Running Two Coding Agents in One Repository

Mick Cooney

5 November 2026

Table of Contents

1 Introduction **4**
- 1.1 What This Primer Covers . . . 4
- 1.2 What This Primer Is Not . . . 5
- 1.3 What You Should Already Know . . . 5

2 Two Harnesses, One Repository **6**
- 2.1 The Model Is a Decision Component, Not the Whole Agent . . . 6
- 2.2 The Harness Supplies the Working Loop . . . 7
- 2.3 The Interface Is Where You Steer the Session . . . 8
- 2.4 Tools and the Execution Environment Define the Agent's Reach . . . 9
- 2.5 Coexistence Is a Repository Practice, Not a Parity Claim . . . 9
  - 2.5.1 A Shared-Repository Operating Model . . . 10
  - 2.5.2 Three Worked Scenarios for Using Both Harnesses . . . 11
- 2.6 A Compact Diagnostic Model . . . 13

3 First-Session Setup **13**
- 3.1 Install and Authenticate Without Changing the Repository . . . 14
- 3.2 Inspect the Repository Before Asking for Edits . . . 15
- 3.3 Choose a Deliberately Small First Task . . . 15
- 3.4 Use a Review Loop, Not a Hand-off . . . 16
- 3.5 End the First Session With a Reproducible Hand-off . . . 17

4 Instruction Systems **17**
- 4.1 Who Owns `AGENTS.md` Now . . . 17
- 4.2 Separate the Repository Contract From the Harness Adapter . . . 18
- 4.3 What `CLAUDE.md` and `AGENTS.md` Mean in This Setup . . . 19
  - 4.3.1 Claude Code and `AGENTS.md` . . . 19
  - 4.3.2 How Claude Code Discovers Instructions . . . 20
  - 4.3.3 How Codex Discovers Instructions . . . 21
- 4.4 The Bridge Pattern, and Which Direction to Point It . . . 23
  - 4.4.1 Point `CLAUDE.md` at `AGENTS.md`, With an Import . . . 23

4.4.2 Pointing `AGENTS.md` at `CLAUDE.md` Is Prose, Not a Mechanism 24
4.4.3 What the Bridge Does Not Do 24
4.5 Shared Guidance Without a Second Manual 24
4.6 Generating Each Harness's Files From One Source 25
4.7 Codex Configuration Is a Supplement, Not the Project Contract 27
4.8 A Practical Instruction Audit 27

5 Workflow Differences **28**
5.1 Exploration: Establish the Map Before Choosing a Route 28
5.2 Planning: Make the Change Boundary Explicit 29
5.3 Implementation: Constrain Scope, Preserve Local Idioms 30
5.4 Testing: Treat Verification as Part of the Task 31
5.5 Review: Review the Diff, Not the Confidence 31
5.6 Sessions, History, Compaction, and Hand-off 32
5.7 Git: Use the Working Tree as the Contract 34
5.8 Parallel Work: Divide by Contract, Not by File Count 35
5.9 A Practical Cross-Harness Loop 36

6 Models and Reasoning Effort **36**
6.1 What the Controls Mean 36
6.2 A Dated Snapshot of the Current Codex Models 37
6.3 Choose by Task Shape 39
6.4 Exploration and Planning Usually Need Less Than Implementation of Ambiguity 40
6.5 A Model-Selection Workflow 40
6.6 Worked Example: Escalating a Repository Task 41
6.7 Fast, Strong, and Specialised Are Not Permanent Categories 42
6.8 Cost and Latency Are Part of Engineering Quality 42
6.9 A Compact Selection Rule 43
6.10 Verification Notes for the Final Edition 43

7 Feature Translation **43**
7.1 Start With the Job, Not the Feature Name 44
7.2 Worked Example: Translating a Claude Code Hook and Skill 44
7.3 Skills: Reusable Workflow Knowledge 49
7.3.1 Skills Are the Closest Thing to a Shared Extension Format 50
7.3.2 Custom Slash Commands Are Being Absorbed Into Skills 51
7.4 Hooks: Automation Around the Agent 51
7.5 MCP: Same Protocol, Different Connection 54
7.5.1 The Configuration Is Not Shared, and There Is No Converter 55
7.5.2 Exposing One Harness to the Other as an MCP Server 56
7.6 Subagents: Delegation Is Not Parallelism by Itself 56
7.6.1 How Codex Subagents Are Defined 57
7.6.2 The Formats Do Not Port, and One Rough Edge Is Unresolved 58
7.7 Permissions and Sandboxing Are Different Controls 59
7.7.1 Codex's Enterprise Layers Have No Claude Code Equivalent 62
7.8 A Migration Rule of Thumb 63

8 Prompting Patterns **63**
8.1 Exploration Before Editing . . . 63
8.2 Bounded Implementation . . . 64
8.3 Review an Existing Diff . . . 65
8.4 Ask for Evidence, Not a Confident Narrative . . . 65
8.5 Prompts for Delegated Work . . . 66
8.6 Recovery Patterns . . . 66
8.6.1 The Agent Changed the Wrong Files . . . 66
8.6.2 The Tests Fail After the Change . . . 67
8.6.3 The Agent Claims a Tool or Capability Is Unavailable . . . 67
8.6.4 The Context Is Stale or Contradictory . . . 67
8.6.5 The Agent Is Over-Scoping the Task . . . 67
8.6.6 The Agent Is Too Hesitant . . . 68
8.7 A Compact Prompt Checklist . . . 68

9 Migration Cookbook **69**
9.1 Move the Canonical Text to `AGENTS.md` and Import It . . . 69
9.2 Choose a Shared-Instruction Strategy . . . 70
9.3 Roll Out in a Branch, One Capability at a Time . . . 71
9.4 Port Integrations by Behaviour, Not by Filename . . . 72
9.5 Preserve a Clean Rollback Path . . . 73

10 Safety, Cost, and Operational Discipline **73**
10.1 Separate Approval From Sandboxing . . . 73
10.2 Protect Secrets and External Systems . . . 74
10.3 Control Cost With Task Boundaries . . . 74
10.4 Make Generated Changes Observable . . . 75
10.5 Establish an Operating Rhythm . . . 75
10.5.1 Safety and Operational Checklist . . . 75

11 What to Measure **76**
11.1 Define the Comparison Before Running It . . . 76
11.2 Use a Task Matrix That Resembles Real Work . . . 76
11.3 Score Correctness Before Speed . . . 77
11.4 Record the Review Burden . . . 78
11.5 Report Distributions, Not a Single Victory Lap . . . 78
11.6 A Practical Evaluation Record . . . 79
11.6.1 Fair Bake-Off Checklist . . . 79

12 Conclusion and Quick Reference **80**
12.1 Quick-Reference: Adding Codex to an Existing Repository . . . 80
12.1.1 Minimal Migration . . . 80
12.1.2 Task-Start Prompt . . . 81
12.1.3 Preflight Checklist . . . 81
12.1.4 Postflight Checklist . . . 81
12.1.5 Choosing a Harness or Model Setting . . . 81

12.1.6 Verification Snapshot and Remaining Volatility 82

13 References **83**

This is a long-form, operational guide for Claude Code users who want Codex as a second coding-agent harness while keeping their existing repository workflow intact.

Product snapshot · 5 November 2026. Instruction discovery, skills, shared-repository workflow, and execution-policy refresh; earlier checks retain their stated dates where they describe historical behaviour.

1 Introduction

If you already use Claude Code, the first few minutes with Codex can be oddly confusing. The terminal looks familiar. Both tools can inspect a repository, edit files, run commands, and explain a diff. You can give both of them a task such as "add a test for this bug" and get something that looks like the same workflow.

Then the similarities stop being useful. The instruction file may have a different name. A command that Claude Code would run immediately may produce an approval request in Codex, or be unavailable in the current execution environment. A model name may describe a different thing from the product or interface you were comparing. A prompt that worked well in one harness may leave the other with too little context about scope, verification, or when it is allowed to edit.

This primer fills that gap. It is a translator's guide for experienced Claude Code users who want Codex as a second coding-agent harness without turning an existing repository into a compatibility project. The goal is not to declare a winner. The useful question is: given this repository, this task, and these constraints, how do you make either harness behave predictably?

1.1 What This Primer Covers

The guide builds a working mental model before it gives you recipes. It explains the layers that people often collapse into the word "Codex" or "Claude," then maps the practical differences that matter when you move between them:

- how a model, an agent harness, a user interface, and tools fit together;
- how Claude Code and Codex can share a repository while keeping their own instruction conventions;

- how to choose a harness and model for a task rather than treating the product name as the decision;
- how to translate familiar concepts such as `CLAUDE.md`, skills, hooks, Model Context Protocol (MCP), subagents, approvals, and sandboxing;
- how to write prompts that state scope, authority, and verification clearly;
- how to inspect, test, and review changes when two agents may work on the same codebase; and
- how to evaluate the two workflows using real tasks instead of anecdotes.

The examples use ordinary repository work: investigating a failing test, changing a build recipe, updating documentation, and reviewing a multi-file refactor. You do not need to adopt every suggested convention. The point is to make the important choices visible.

1.2 What This Primer Is Not

This is not a complete reference manual for either product. Commands, configuration keys, available models, plan entitlements, usage limits, and interface behaviour can change. Where an exact product detail matters, treat it as a dated claim and check the current official documentation before relying on it in automation or team policy.

It is also not a feature-by-feature promise that every Claude Code capability has a Codex equivalent, or vice versa. Similar labels do not guarantee similar semantics. A "skill," "hook," or "subagent" may be implemented differently, may require a different configuration, or may have no direct counterpart. The safe approach is to compare the behaviour you need, such as "run this check before committing" or "delegate this independent investigation," and then find the mechanism that provides that behaviour in the chosen harness.

Finally, this guide does not ask you to rewrite a working Claude Code setup. Claude Code can remain the primary harness. One canonical instruction file can serve both agents, while shared build commands, tests, and neutral project documentation remain shared assets. You should only split instructions or workflows when the difference is real and worth maintaining. The chapter on instruction systems explains which file should hold that canonical text and why the direction of the bridge between `CLAUDE.md` and `AGENTS.md` is not an arbitrary choice.

1.3 What You Should Already Know

You should be comfortable working in a terminal, reading a git diff, running a project's tests, and making ordinary changes across several files. You should know what your repository's `CLAUDE.md` says and understand the purpose of the tools it mentions. Familiarity with skills,

hooks, MCP, and agentic coding will make the translations shorter, but the guide introduces each term when it becomes relevant.

You do not need to know Codex's configuration syntax in advance. You do need to bring the habits that make any coding agent safe to use: give it a bounded task, keep the working tree understandable, inspect its proposed changes, and run the checks that establish whether the result is correct.

Verified against primary sources for all sections; unverified claims are flagged inline.

2 Two Harnesses, One Repository

The most useful starting point is to stop comparing "Claude" and "Codex" as if each were a single interchangeable object. In a coding session, at least four layers are involved:

1. the **model**, which generates text and tool-use decisions;
2. the **agent harness**, which maintains the task loop and turns those decisions into work;
3. the **interface**, through which you start the session and observe or steer it; and
4. the **tools and execution environment**, which determine what the agent can inspect or change.

These layers interact, but they are not synonyms. Two sessions can use the same underlying model and still behave differently because the harness supplies different instructions, exposes different tools, or applies different approval rules. Conversely, the same harness can behave differently after you change the model, reasoning setting, working directory, or sandbox.

2.1 The Model Is a Decision Component, Not the Whole Agent

The model is the language-and-reasoning component that predicts the next response or action from the context it receives. In a coding task, that context can include your prompt, repository instructions, files, command output, previous tool results, and the harness's own rules. The model can propose "inspect the test," "edit this function," or "run the test suite," but it does not by itself define how those actions happen.

For example, suppose you ask:

```
The parser accepts a trailing comma in arrays but rejects one in objects.
Find the cause and add a regression test.
```

The model may infer a sensible investigation: locate the parser, compare the two grammar paths, reproduce the failure, and inspect related tests. The harness decides how to expose the filesystem and shell, whether the model may edit immediately, how to present command

output, and when to ask for approval. If the session has no way to run the test suite, a capable model cannot manufacture that missing capability.

Model choice still matters. Different models can vary in coding accuracy, speed, context handling, and ability to sustain a multi-step investigation. Reasoning controls can also change how much effort a model spends before acting. Those are important choices, but they are only one part of the observed workflow. A strong model in a poorly scoped session can make a larger mess than a less powerful model given clear boundaries and a reliable verification loop.

Exact model names, availability, limits, and pricing are volatile product facts. Do not bake an undated model table into repository instructions. Keep task-level guidance such as "use a faster model for a small formatting change; use a deeper reasoning setting for an unfamiliar concurrency bug" separate from the current product catalogue. The model-selection chapter will return to this distinction.

2.2 The Harness Supplies the Working Loop

The agent harness is the program that turns a conversation into an agentic coding session. It loads instructions, assembles context, presents tools, interprets the model's requested actions, applies permission checks, records results, and keeps the loop moving until the task is complete or it needs your input.

Claude Code and Codex are different harnesses. They overlap substantially: both can be used for repository exploration, implementation, testing, and review. That overlap lets you move familiar tasks between them. It does not imply identical defaults or feature semantics.

Imagine that both harnesses receive this request:

```
Inspect the build failure, explain the root cause, and propose a fix. Do not edit files yet.
```

One harness may naturally spend the session in an investigation-and-plan phase. Another may inspect the failure and then ask whether it should apply the proposed patch. Both outcomes can be reasonable. Your prompt should make the important boundary explicit, and you should verify that the harness honoured it before proceeding.

The harness also determines how repository guidance is discovered, and this is the first place where the two tools genuinely disagree rather than merely differ. Claude Code and Codex can both read `AGENTS.md`, but they do not choose or combine instruction files in the same way. A project `CLAUDE.md` normally takes precedence in Claude Code, while Codex walks an `AGENTS.md` chain and supports `AGENTS.override.md`. Putting operational rules in two files without testing the effective instruction chain can therefore produce sessions with different contracts, even when both agents appear to have loaded project guidance.

The safest default is one canonical file plus a real include or an explicitly configured both-files policy, not two manuals. Claude Code supports an `@path` import, so a one-line `CLAUDE.md` can pull in the whole of `AGENTS.md`:

```
@AGENTS.md
```

That is a resolved import rather than a suggestion to the model, which is what makes it reliable. The reverse arrangement — a prose `AGENTS.md` linking to `CLAUDE.md` — is not equivalent, because Codex reads the contents of the file it discovers and does not treat a Markdown link inside it as an include directive. The instruction-systems chapter covers both directions, the supported Codex configuration key for the reverse case, and what each one actually guarantees.

This still does not claim that Codex and Claude Code will interpret every instruction identically. It gives both sessions the same text, while leaving room to state a real difference when one exists. For instance, an instruction about a particular approval mode belongs with the harness that implements that mode; a command for building the project belongs in the shared canonical file.

2.3 The Interface Is Where You Steer the Session

The interface is the surface you use to start, observe, and control the harness. It might be a command-line session, an editor integration, or another product surface. The interface affects discoverability and ergonomics: how you select a model, ask for a plan, see a diff, resume a session, or respond to an approval request.

Do not infer execution capability from interface appearance. A command-line interface may still be operating inside a restricted sandbox. An editor panel may show a plan without granting permission to write. A session started with one product surface may have different authentication, network access, or model availability from a session started with another. When a task behaves unexpectedly, identify which interface launched the harness before blaming the model.

As a concrete example, "run the integration tests" can fail for several unrelated reasons:

- the model chose the wrong command;
- the harness could not find the repository's test instructions;
- the interface started the session in the wrong working directory;
- the execution environment lacks a service, dependency, or network route; or
- an approval or sandbox rule prevented the command from running.

The final error may look similar in the conversation, but the fixes are different. Ask which layer failed.

2.4 Tools and the Execution Environment Define the Agent's Reach

Tools are the capabilities the harness exposes to the model: reading files, searching text, applying patches, running shell commands, inspecting git state, calling an external service, or interacting with a browser. The execution environment is where those tools run and what they can reach. It includes the working directory, installed dependencies, containers, environment variables, network policy, credentials, and filesystem restrictions.

This distinction matters because "the agent can use shell" is not a complete statement. Shell access might mean read-only commands, commands that run only after approval, or commands inside a sandbox with no network and a limited filesystem. A tool may be present but unable to reach a service. A repository instruction may describe a container command that is valid on one machine and unavailable on another.

Keep two questions separate:

- **Approval:** may this proposed action proceed? An approval request is a control decision about a particular action.
- **Sandbox:** what can the execution environment permit even if the action is approved? A sandbox can still block network access, writes outside the workspace, or access to a required service.

For example, if Codex asks for approval to run a package-install command, approving it does not necessarily mean the command can write to every system directory or reach the package registry. Conversely, a permissive sandbox does not mean the harness will run every command without asking. Confusing these concepts leads to bad debugging and unsafe assumptions.

2.5 Coexistence Is a Repository Practice, Not a Parity Claim

Claude Code and Codex can work on the same repository because the repository already contains the most important shared assets: source code, tests, build commands, version history, and project conventions. They do not need identical internal workflows to share those assets.

The safe coexistence pattern is deliberately modest:

1. Keep project facts and shared conventions in a canonical location.
2. Give each harness a clear instruction-file entry point.
3. Put only real harness-specific behaviour in harness-specific guidance.
4. Ask the agent to inspect the working tree before making a broad change.
5. Review the diff and run project checks after every meaningful task.

Suppose Claude Code has just added a database migration and left the working tree with a migration file, a schema change, and tests. You open Codex and ask it to "finish the migration." That phrase is underspecified. Codex may create a second migration, alter the existing one, or focus on tests while missing the deployment note. A safer handoff names the current state and the allowed scope:

```
Review the current working tree after another agent's database migration work.
Do not rewrite the existing migration. Identify missing tests or documentation,
make only the smallest changes needed to complete the task, and run the
migration-specific checks. Report any uncertainty before changing schema files.
```

The repository is shared; the session state is not necessarily shared. Do not assume that Codex knows what Claude Code discussed earlier, that it has the same context window, or that it will interpret an informal handoff the same way. The durable handoff is the working tree, the git diff, the tests, and a short written note that states what remains.

Coexistence also means accepting asymmetry. One harness may be more convenient for a particular tool integration, permission policy, or long-running workflow. One model may be better suited to a particular investigation. One environment may have the credentials or services the task requires. Use the mechanism that fits the task, and make the result legible to the other harness through files, commands, tests, and explicit notes.

2.5.1 A Shared-Repository Operating Model

There are three different meanings of "use both tools in one repository", and they need different controls:

Situation	Safe default	Why
One agent finishes, then the other continues in the same checkout	Serialise the sessions; inspect `git status` and the diff at every hand-off	The working tree is the shared state, so the second agent can see exactly what changed
Both agents work at the same time on one task	Use separate git worktrees and branches, with one integration owner	Two processes can edit the same path or make incompatible assumptions before either sees the other's changes
One agent implements while the other reviews	Give the reviewer a clean worktree or a committed/refreshed branch and make review read-only	A reviewer who edits while reviewing is no longer independent, and concurrent edits obscure the evidence

Do not run Claude Code and Codex simultaneously in the same checkout merely because both can read the files. Git does not lock ordinary source files, and neither harness provides a general-purpose cross-tool ownership protocol. The failure mode is not only a textual conflict: one agent can make a valid change against an old interface while the other is changing that interface, so both patches may apply cleanly and still be wrong together.

When concurrency is worthwhile, create one branch and worktree per agent. Give each worker an explicit ownership boundary, such as "investigate and test the API, but do not edit" or "change only `src/payments/` and its tests". Keep generated files, lockfiles, migrations, and shared interfaces under one owner unless you have already agreed how to integrate them. A worktree isolates files and branch state; it does not resolve design disagreement or make shared services safe to mutate.

Use a small hand-off record when the work spans sessions. It can be an issue comment, pull-request description, or temporary note excluded from the final artefacts, but it should name the starting revision, changed paths, checks that ran, known failures, and the next permitted action:

```
Base: 7f31c2a
Owner: Claude Code, feature branch
Changed: src/payments/expiry.py, tests/test_expiry.py
Verified: pytest tests/test_expiry.py (passed)
Not verified: integration tests require the local database
Next agent: review the boundary condition; do not rewrite the public API or
run deployment commands.
```

The hand-off is deliberately factual. Do not write "Claude says this is done" or "Codex checked everything". Write the revision, paths, commands, and results so either harness can reproduce the claim. If the next agent cannot reconstruct the state from the record and the checkout, the hand-off is too dependent on conversational memory.

Finally, distinguish shared *content* from shared *discovery*. A `SKILL.md` can often be shared because both tools understand the Agent Skills format, but Claude Code and Codex discover project skills in different directories. Custom subagents, hooks, MCP configuration, and permission rules remain harness-specific even when they implement the same workflow. A successful co-use setup therefore has one repository contract, a tested skill-distribution strategy, separate adapters where needed, and an explicit rule for who owns each live checkout.

2.5.2 Three Worked Scenarios for Using Both Harnesses

The following scenarios turn the principles into an operating sequence. They are intentionally tool-neutral at the repository level: the commands establish state, while the harness-specific prompt controls how Claude Code or Codex acts on that state.

Sequential hand-off in one checkout. Claude Code implements a focused change and stops without committing. Codex then reviews and completes it:

```
git status --short
git diff --name-status
git diff -- src/payments tests/test_payments.py
```

The first agent records the starting revision, owned paths, changed paths, exact checks and their outcomes, and the remaining uncertainty. The second agent must inspect those facts before editing, preserve unrelated changes, and stop if the working tree no longer matches the hand-off. If Claude Code has changed an interface after Codex started from an older branch, Codex should report the stale assumption and re-plan or rebase; it should not silently repair the mismatch as if it were part of the original task.

Concurrent implementation in isolated worktrees. A coordinator assigns Claude Code the API implementation and Codex the independent test fixture:

```
git worktree add ../repo-claude -b agent/claude-api main
git worktree add ../repo-codex -b agent/codex-fixture main
git -C ../repo-claude status --short
git -C ../repo-codex status --short
```

The coordinator reserves shared interfaces, migrations, lockfiles, and generated outputs rather than assigning them to both workers. Each agent returns a diff and verification record. The coordinator integrates the branches, resolves semantic conflicts, runs the combined checks, and reviews the result. Once the integration is complete, remove only the disposable worktrees after checking that no uncommitted work remains:

```
git worktree remove ../repo-claude
git worktree remove ../repo-codex
git worktree prune
```

Worktrees prevent accidental file overwrites; they do not make two agents independent if they both change the same contract or mutate the same external database. Keep one owner for shared state and one human or coordinating agent responsible for integration.

Implementation followed by an independent review. The implementation agent commits or refreshes a branch after running its focused checks. The reviewer opens a clean worktree at that revision, receives a read-only task contract, and reports findings with paths and evidence. The reviewer must not fix the diff while claiming independence. After the implementation owner addresses the findings, the reviewer examines the refreshed diff or a new commit. This two-pass pattern is more informative than opening a second chat in the same dirty checkout, where the second agent may inherit unobserved state and its edits become part of the thing it was supposed to assess.

A reusable hand-off record is small enough to keep in an issue or pull request:

```
Starting revision: <commit or branch>
Owned paths: <paths this agent may change>
Changed paths: <actual paths changed>
```

```
Checks and exact outcomes: <commands and results>
Remaining uncertainty / next allowed action: <explicit boundary>
```

This is also the pattern that recurs in current practitioner material: keep a shared `SKILL.md` procedure but account for separate discovery directories, use a canonical `AGENTS.md` or an explicit import rather than copy-and-paste instruction files, and isolate simultaneous workers with worktrees. Treat those reports as experience reports rather than standards, but they are useful evidence that the operational risks are real rather than hypothetical; see the NICAR Codex/Claude skills materials and the Codex/Claude bridge implementation for representative examples.

2.6 A Compact Diagnostic Model

When a session goes wrong, classify the failure before changing prompts or models:

```
model       -> reasoning, code generation, action proposals
harness     -> context assembly, tool loop, instructions, approvals
interface   -> session controls, visibility, selection, continuation
tools/env   -> files, commands, services, credentials, execution limits
repository  -> source, tests, build rules, durable handoff state
```

If the generated patch is logically wrong, inspect the model's reasoning and the context it received. If the agent ignored a repository rule, inspect instruction discovery and harness behaviour. If it proposed the right command but could not run it, inspect tools, approval, and sandbox constraints. If the second agent misunderstood the first agent's work, inspect the diff and handoff rather than assuming a model upgrade will solve the coordination problem.

This layered model is the foundation for the rest of the primer. The next sections turn it into an initial setup, an instruction-file strategy, and task workflows that let you switch harnesses without losing control of the repository.

3 First-Session Setup

Your first Codex session should not begin with a large feature request. The useful first question is whether Codex can understand the repository, respect its boundaries, and leave you with a diff you can explain. Treat the first session as a controlled integration test for a second harness.

The exact installation, authentication, model, and configuration commands are product details that change. The examples in this section use the command name `codex` where an executable is needed; check those commands against the current official Codex documentation before you publish or automate them. This guide intentionally avoids turning a volatile login flow into repository policy.

3.1 Install and Authenticate Without Changing the Repository

Install Codex using the current supported path for the interface you intend to use: command-line, editor integration, or another supported client. Keep the installation and authentication step separate from repository changes. You want to be able to tell whether a later problem comes from the client, the account, the repository instructions, or the task itself.

The following commands are Codex CLI examples, not a universal installation recipe for every Codex surface. After installation, use the client's built-in help or diagnostics before opening a real task. This confirms which commands and flags exist in the version you actually installed.

```
# The executable name and available flags can change between Codex releases;
# consult the current official documentation if this command differs.
codex --help
```

The CLI, IDE extension, ChatGPT desktop app, ChatGPT web experience, and Codex cloud are related surfaces with different controls. The CLI exposes commands such as `codex`, `codex exec`, `/model`, `/permissions`, and `/review`; an IDE or desktop surface exposes equivalent choices through its own controls, while cloud work runs in a hosted environment. Use the Codex CLI guide for terminal commands and the documentation for the specific surface when the execution environment matters.

Authentication may be browser-based, environment-variable-based, or managed by the host application. Do not put credentials in `CLAUDE.md`, `AGENTS.md`, `.codex/config.toml`, prompts, or committed shell scripts. If the repository needs a service token for tests, use the project's existing secret-management procedure and confirm that the token is not included in command output or the diff.

Before the first write-capable session, establish what the client is allowed to do. "Approval" means whether you permit a proposed action. "Sandbox" means what the execution environment permits even if the action is approved. They are related, but they are not the same control. A command can require approval, and an approved command can still fail because the sandbox blocks network access, a filesystem path, a container runtime, or another capability.

The names and defaults for these controls vary by Codex interface and release. Treat any command-line flags, configuration keys, or UI labels as provisional until checked against the version in use. Start with the narrowest permissions that let the task run. Widen them only when a specific, understood operation needs it.

3.2 Inspect the Repository Before Asking for Edits

Open Codex from the repository root, or explicitly tell it which directory is the project root. The root matters because instruction files, build commands, relative paths, and git state are all interpreted in relation to the project.

Do a human inspection first. You need to know whether the worktree is clean, which files are already modified, and where the project describes its normal build and test commands. A useful baseline is:

```
git status --short
git branch --show-current
rg --files \
  -g 'CLAUDE.md' -g 'AGENTS.md' -g 'README*' -g 'Justfile' \
  -g 'Makefile' -g 'package.json' -g 'pyproject.toml' | sort
```

The `rg` command is a discovery aid, not a universal project contract. Use the build system the repository already documents. Do not create a new package manager, test runner, or formatting convention merely because Codex knows one.

Your first prompt should request inspection only. Ask for the repository layout, applicable instructions, likely validation commands, and any concerns about the current state. Make the no-edit boundary explicit:

```
You are in the repository root. Do a read-only orientation pass.

Inspect the applicable instruction files, the top-level README, the relevant
build/test configuration, and the current git status. Do not edit files, run
destructive commands, install dependencies, commit, or contact external
services.

Report:
1. Which instruction files you found and which appear to apply.
2. The repository's main build, test, lint, and formatting commands.
3. The files and directories relevant to the next small documentation task.
4. Any dirty-worktree changes you will leave untouched.
5. Assumptions or missing information that should be resolved before editing.
```

This prompt tests more than file listing. It tests whether the agent can identify instructions as instructions, distinguish existing changes from its own work, and state uncertainty instead of silently filling gaps. If the answer invents a test command, claims that it changed nothing without showing how it checked, or misses an obvious instruction file, stop and fix the setup before giving it write access.

3.3 Choose a Deliberately Small First Task

The first write should be reversible and easy to review. Updating one sentence in a document, adding a focused test, or correcting a narrow configuration comment is usually better than beginning with a cross-cutting refactor. The task should exercise the repository's normal workflow without making the first session depend on every tool integration at once.

Give Codex a bounded objective, explicit exclusions, and a validation request. For example:

```
Make one narrowly scoped documentation change: clarify the sentence that
describes how the project builds its HTML output.

Constraints:
- Read the applicable instruction files before editing.
- Edit only the relevant Markdown source and nothing else.
- Preserve the existing voice and heading structure.
- Do not update generated HTML/PDF files in this task.
- Do not change build configuration, dependencies, or unrelated wording.

After editing, show the diff, run the cheapest relevant validation command if
one is documented, and report anything you could not verify. Do not commit.
```

The exclusions are important. A coding agent often sees generated files, formatting opportunities, stale documentation, or a failing unrelated test and tries to make the repository "better" while it is there. That may be useful on a later task, but it makes the first diff harder to attribute. Scope discipline is part of what you are testing.

3.4 Use a Review Loop, Not a Hand-off

The basic loop is:

```
inspect -> propose -> edit -> validate -> review -> accept or revise
```

Do not treat a successful tool call as evidence that the result is correct. Review the diff yourself. At minimum, check the changed paths, the actual content, whitespace, generated files, and the validation output.

```
git status --short
git diff --check
git diff -- path/to/the/expected/file.md
```

Then run the repository's documented validation command. For a small Markdown change that might be a link checker or a render preview; for code it might be the focused test followed by the normal test suite. A command that is not documented, that installs software, or that reaches a network service should be treated as a separate decision rather than an automatic part of the loop.

Ask Codex to explain the result in terms of evidence, not confidence:

```
Review the current diff against the original request.

Report:
- every file changed;
- which requested constraints you verified and how;
- commands run and their exit status;
- checks you could not run and why;
- any behaviour or wording that remains uncertain.

If the diff exceeds the requested scope, propose a correction before making
more edits. Do not commit.
```

If the result is wrong, give a targeted correction and keep the same review loop. Do not respond to an incorrect first attempt by broadening the task. A small, explicit correction tells you whether the agent can recover without losing the original constraints.

3.5 End the First Session With a Reproducible Hand-off

Before you close the session, ask for a short hand-off containing the final state, validation performed, unresolved questions, and the next safe action. You should be able to resume the work from the repository and the diff rather than relying on a vague memory of the conversation.

```
Summarize this session for another engineer:
- the original objective;
- files changed and why;
- validation commands and results;
- assumptions and unresolved issues;
- any follow-up work that is explicitly out of scope.

Do not modify files or commit while writing the summary.
```

A first session has gone well when you know what Codex did, what it did not do, which instructions it followed, and which parts still need human judgement. It has not gone well merely because the client started, a model responded, or the task ended without an error message.

4 Instruction Systems

Claude Code and Codex both use project instructions, but an instruction file is not a portable command language. `CLAUDE.md` and `AGENTS.md` are conventions used by different harnesses; they do not automatically have identical discovery rules, precedence, inheritance, or feature support. The practical problem is to give two agents one repository contract without maintaining two conflicting manuals.

4.1 Who Owns `AGENTS.md` Now

Both of the conventions in this chapter stopped being single-vendor conventions on 9 December 2025. On that date the Linux Foundation announced the formation of the Agentic AI Foundation (AAIF), and contributed as founding projects were `AGENTS.md`, which originated at OpenAI, and the Model Context Protocol (MCP), which originated at Anthropic. Block's `goose` agent was the third founding project. The announcement lists platinum members including Anthropic, OpenAI, Amazon Web Services (AWS), Google, Microsoft, Block, Bloomberg, and Cloudflare.

That matters for a two-harness repository in a specific way. The instruction filename you are standardising on is no longer owned by the vendor of one of your two agents, so a bet on `AGENTS.md` is not a bet on OpenAI. Claude Code now has built-in `AGENTS.md` support in recent releases, but its default selection rule is conditional: a project `CLAUDE.md` normally takes precedence, so the presence of both files can still produce different instruction sets. The next subsection explains the current rule and the explicit settings that make a shared arrangement predictable.

`AGENTS.md` has no schema. The `agents.md` site is explicit that the file is "just standard Markdown" with no required fields, and the sections it recommends — build and test commands, code style, testing instructions, security notes, commit and pull-request conventions — are convention only. Nothing validates your file, nothing warns you when a harness ignores a section, and two harnesses that both claim to read `AGENTS.md` can still weight its contents differently. Portability here is about the filename and the discovery algorithm, not about a guaranteed interpretation of the text.

Adoption is broad on paper. The `agents.md` site lists more than 26 tools as native adopters, including Codex, Cursor, Aider, Google Jules, GitHub Copilot, Windsurf, Gemini CLI, Zed, Warp, JetBrains Junie, and Devin, and claims more than 60,000 repositories containing the file. Both figures are self-reported by the site rather than independently audited, so treat the repository count as a marketing number. The tool list is the more useful half: it tells you which other agents will find a root `AGENTS.md` without extra configuration, which is exactly the question a repository owner needs answered before choosing where the canonical text lives.

4.2 Separate the Repository Contract From the Harness Adapter

Start by classifying an instruction:

Kind of guidance	Examples	Best home
Repository-wide	build commands, test requirements, source/output rules, coding style, generated-file policy	The canonical instruction file, which both harnesses should read in full
Claude Code-specific	Claude-only skills, hooks, commands, or interaction conventions	`CLAUDE.md` below the import, or `.claude/` configuration

Kind of guidance	Examples	Best home
Codex-specific	Codex approval policy, sandbox mode, subagent definitions, or session controls	`.codex/config.toml`, not the shared instruction file
Task-specific	Constraints for one migration, experiment, or directory	A per-directory `AGENTS.override.md`, or the task prompt

The most important distinction is between a policy and an adapter. "Run the focused test before declaring success" is repository policy. "Use this harness's skill command to discover a workflow" is an adapter detail. Keep the policy stable and make each harness adapter as thin as possible.

Do not copy a large `CLAUDE.md` into `AGENTS.md` just to make both agents see the same words. Duplication creates two sources of truth. The files will drift, and the drift will be hardest to notice when the instructions concern safety, generated outputs, or validation.

4.3 What `CLAUDE.md` and `AGENTS.md` Mean in This Setup

For this guide, use these terms precisely:

- `CLAUDE.md` is the file Claude Code discovers. In the arrangement this chapter recommends it holds an import of the canonical text plus any Claude-specific additions, not the canonical text itself.
- `AGENTS.md` is the file Codex discovers, and the better home for the canonical repository contract because more tools read it and because Codex supports per-directory overrides of it.
- "Instruction file" is the neutral term when the advice applies to either harness.
- A referenced file is not necessarily an included file. `@path` in `CLAUDE.md` is a resolved import. A Markdown link is not. That distinction decides which direction a bridge can safely point.

4.3.1 Claude Code and `AGENTS.md`

This was the open question in earlier drafts of this primer. Anthropic's current memory documentation (`code.claude.com/docs/en/memory`, checked 5 November 2026) says that Claude Code reads `AGENTS.md` when there is no `CLAUDE.md`, `.claude/CLAUDE.md`,

or `CLAUDE.local.md` in the working directory or an ancestor. The default is therefore `CLAUDE.md` *or* `AGENTS.md`, not both. A repository that contains both files can still give Claude Code a different contract from Codex without any model failure: Claude Code may load the `CLAUDE.md` chain while Codex loads `AGENTS.md`.

Recent Claude Code releases also expose a Project instructions setting that can load `CLAUDE.md` and `AGENTS.md` together. That is useful when a team is migrating, but it does not make duplicate prose safe: conflicting instructions remain conflicting instructions, and an `AGENTS.md` loaded through the setting does not have exactly the same hook behaviour as a file imported by `CLAUDE.md`.

There are now three sensible arrangements. The first is the explicit one-line `CLAUDE.md` import:

```
@AGENTS.md
```

`@path` is Claude Code's native import syntax, not a Markdown link and not a hint to the model. Claude Code resolves the import when it assembles context, so the contents of `AGENTS.md` are genuinely in the session. The second is a symlink:

```
ln -s AGENTS.md CLAUDE.md
```

Prefer the `@AGENTS.md` import when portability and Claude-specific additions matter. The symlink is one fewer file's worth of content, but symlinks on Windows require Developer Mode or an elevated shell to create, and a repository that any contributor may clone on Windows should not depend on one. The import file is plain text that works identically everywhere, and it leaves room to append genuinely Claude-specific lines below the import without disturbing the shared text.

The third is the current Project instructions setting, which can tell Claude Code to load both files. Use it when the repository genuinely needs a Claude-specific adapter alongside shared guidance; do not use it as an excuse to keep two copies of the same policy. With `CLAUDE_CODE_NEW_INIT=1`, `/init` can inspect `AGENTS.md` and other tools' rule files. `/import` can perform a one-time copy of another agent's configuration — including instruction files, MCP servers, commands, subagents, and skills — into Claude Code's native locations. These are migration aids, not live synchronisation: later edits do not propagate. For a repository where both harnesses are in continuous use, the import or a deliberate both-files setting is easier to audit than an unexplained pair of drifting manuals.

4.3.2 How Claude Code Discovers Instructions

Claude Code loads instruction files in this order, and the order is concatenation rather than override:

1. an enterprise-managed policy file, deployed by an organisation's IT administrators, which a user cannot exclude;
2. the user file at `~/.claude/CLAUDE.md`, which applies to every project;
3. project files, meaning `CLAUDE.md` or `.claude/CLAUDE.md` in the repository, discovered from the filesystem root down to the current working directory; `AGENTS.md` files are also discovered when no project `CLAUDE.md` takes precedence, or when Project instructions is configured to load both; and
4. `CLAUDE.local.md`, a personal file intended to be gitignored.

Every discovered file is included. A project `CLAUDE.md` does not replace the user file, and a deeper file does not replace a shallower one. Files are concatenated in root-to-cwd order, so guidance closer to the working directory appears later in the assembled context and is the practical tie-breaker when two files disagree — but the earlier text is still present, and a contradiction between the two is a contradiction the model has to resolve, not a resolved precedence decision. Write ancestor files to be additive rather than relying on a descendant to cancel them.

`CLAUDE.md` files in nested subdirectories behave differently from the ones on the root-to-cwd path: they load on demand, when work reaches that directory, rather than at launch. That keeps the startup context small in a large tree, and it means a rule buried three directories down may not be in context when the agent decides how to approach the task. Put anything that must always apply on the root-to-cwd path.

For monorepos, the `claudeMdExcludes` setting lets you skip specific ancestor files. That is the lever for the case where a repository sits inside a parent directory whose `CLAUDE.md` describes an unrelated project, or where an organisation-wide file is accurate but too long to be worth the context it consumes on every session.

4.3.3 How Codex Discovers Instructions

Codex's algorithm is genuinely different, and more monorepo-native. According to OpenAI's `AGENTS.md` guidance (checked 10 September 2026), Codex resolves a global file first — `~/.codex/AGENTS.override.md` if it exists, otherwise `~/.codex/AGENTS.md` — and then walks from the project root down to the current working directory. In each directory it checks for `AGENTS.override.md` first, then `AGENTS.md`, then any configured fallback names, and includes at most one file per directory. The search stops at the current working directory and does not descend below it. The discovered files are concatenated root-to-cwd, so a file closer to the working directory appends after, and effectively overrides, the guidance above it. The combined size is capped by `project_doc_max_bytes`, which defaults to 32 KiB.

The `.override.md` variant is the part with no Claude Code equivalent. It gives you a real per-subtree override: `services/payments/AGENTS.override.md` wins over `services/payments/AGENTS.md` for any work whose working directory is in that subtree, without touching the root file that governs everything else. In a monorepo where one service has a different test runner, a different deployment constraint, or a stricter change policy than the rest of the tree, that is the mechanism you want, and it is the strongest argument for putting the canonical text in `AGENTS.md` and importing it into `CLAUDE.md` rather than the reverse.

The two algorithms side by side:

Question	Claude Code	Codex CLI
Filenames read	`CLAUDE.md` chain by default; `AGENTS.md` when no project `CLAUDE.md` takes precedence or when configured to load both; managed policy file	`AGENTS.override.md`, then `AGENTS.md`, then configured fallbacks
User-level file	`~/.claude/CLAUDE.md`	`~/.codex/AGENTS.override.md`, else `~/.codex/AGENTS.md`
Directory walk	Root to cwd, all matches included	Root to cwd, at most one file per directory
Below cwd	Nested files load on demand during work	Not searched
Combination rule	Concatenated, root to cwd	Concatenated, root to cwd
Per-subtree override	None; use `claudeMdExcludes` to drop an ancestor	`AGENTS.override.md` in that directory
Size cap	Not documented as a byte limit	`project_doc_max_bytes`, default 32 KiB
Import primitive	`@path` imports are resolved	Discovery only; a Markdown link is not an include
Reads the other tool's file	Yes, conditionally or through `@AGENTS.md`; `/init` and `/import` can also copy it	Yes, via `project_doc_fallback_filenames`

The last row is no longer a complete description of Claude Code. Recent Claude Code releases can read `AGENTS.md` directly, but the default is conditional and older or restricted sessions

may still require the import. Codex's fallback key remains useful when a repository has only `CLAUDE.md`; it is not a live bridge and does not solve a repository that also contains `AGENTS.md`.

The same caution applies to configuration keys as to product behaviour generally. A key accepted by one Codex release may be renamed, moved, or removed in another, and Claude Code's settings schema moves too. Configuration should support the instruction design; it should not be the only place where a safety or validation rule exists.

4.4 The Bridge Pattern, and Which Direction to Point It

A bridge file gives the second harness an entry point to the first harness's instructions. There are two directions to point it, and they are not equally reliable. Getting this right is the single most consequential decision in the chapter.

4.4.1 Point `CLAUDE.md` at `AGENTS.md`, With an Import

The most explicit bridge uses a real include primitive. Put the canonical instructions in `AGENTS.md` and make `CLAUDE.md` an import of it:

```
@AGENTS.md
```

Claude Code resolves `@AGENTS.md` when it assembles context, so the full text is present in the session. Codex finds `AGENTS.md` by its own discovery walk. One file holds the repository contract, both harnesses receive it, and the arrangement remains explicit even when Claude Code's default discovery rule changes. If Claude Code needs extra instructions that mean nothing to Codex, append them below the import:

```
@AGENTS.md

### Claude Code Specifics

- Use the `render-primer` skill for build work; it wraps the container recipe.
- The pre-commit hook in `.claude/settings.json` blocks committed intermediates.
```

This remains a useful arrangement when a repository already has a `CLAUDE.md`, when Claude Code sessions are on a release that cannot load `AGENTS.md`, or when the team wants the bridge to be visible in the tree. It also makes Claude-specific additions easy to isolate below the import. Do not describe it as proof that Claude Code cannot read `AGENTS.md`: current releases can do so under the documented conditions above.

The symlink alternative, `ln -s AGENTS.md CLAUDE.md`, produces the same result with no file content at all. It is fine on Linux and macOS. It needs Developer Mode or an elevated shell on Windows, and a repository that Windows contributors may clone should use the import instead.

4.4.2 Pointing `AGENTS.md` at `CLAUDE.md` Is Prose, Not a Mechanism

Earlier drafts of this primer recommended the reverse: keep `CLAUDE.md` canonical and add a thin `AGENTS.md` that links to it. That arrangement looks symmetrical and is not. Codex discovers `AGENTS.md` and reads its contents; it does not treat a Markdown link inside that file as an include directive. What the agent receives is a short document telling it that the real instructions are somewhere else. Whether it then reads `CLAUDE.md` depends on whether the model decides to, which is exactly the kind of dependency an instruction system should not have.

Codex does offer a configuration key for this case, and it is a better tool than the prose link:

```
# ~/.codex/config.toml or <repo>/.codex/config.toml
project_doc_fallback_filenames = ["CLAUDE.md"]
```

That makes `CLAUDE.md` a discovered instruction file in its own right when `AGENTS.md` is absent from a directory. Note the limits. It is a fallback, so it does not apply when `AGENTS.md` exists in the same directory; it lives in Codex configuration rather than in the repository contract, so a contributor who has not set it gets nothing; and the project-local `.codex/` layer is skipped entirely for untrusted projects, which makes trust status part of your instruction design rather than an implementation detail.

If you have an existing repository with a substantial `CLAUDE.md` and you do not want to move it, the fallback key plus a prose `AGENTS.md` is a defensible interim state. It is not the arrangement to build a team convention on: it depends on every Codex user having the same local configuration. Prefer a single canonical file, or make the both-files arrangement explicit and test it in both harnesses.

4.4.3 What the Bridge Does Not Do

A bridge establishes where a repository-level reader should begin. It is not a parity claim. Skills, hooks, MCP servers, subagent definitions, approval policy, and sandbox policy all live in harness-specific files with different formats, and the rest of this primer covers each of them. Test the bridge the same way you would test any other assumption: open a read-only session in each harness and ask which instruction files it found and which rules it believes apply. An agent that cannot name your build command has not read your instructions, whatever the file tree looks like.

4.5 Shared Guidance Without a Second Manual

There are three reasonable designs. Pick one deliberately instead of allowing the repository to accumulate all three.

The first is the import bridge: put the canonical text in `AGENTS.md` and make `CLAUDE.md` a one-line `@AGENTS.md` import. This is the smallest change that gives both harnesses the same instructions with no dependency on a model following a link, and it is the right default in almost every case. Keep the importing file free of rules that exist nowhere else, so that a reader of `AGENTS.md` alone is never missing a constraint.

The second is a neutral shared document with small adapters. For example, a repository could keep build and validation policy in `PROJECT_GUIDE.md`, then have each harness instruction file tell its agent to read that guide. This is clean when both harnesses are first-class and the shared contract is large. It adds another file to discover, so the first-session inspection must verify that both agents read it.

The third is a deliberately duplicated minimal contract. Each harness file contains the same short list of non-negotiable rules, while the detailed tool-specific guidance stays separate. This can be appropriate when a harness cannot reliably follow references, but it creates a maintenance obligation. Keep the duplicated section short and make drift detectable in review.

For this project, the second design is now the repository pattern. The shared contract lives in `docs/project-contract.md`, and `AGENTS.md` is the canonical root entry point. The contract also routes to `docs/agent-collaboration.md` for handoffs and smoke tests. Claude-specific settings may remain under `.claude/`, but they are not a second repository policy manual. This keeps repository policy neutral without pretending that native skills, hooks, permissions, or subagent formats are portable.

4.6 Generating Each Harness's Files From One Source

Two harnesses is manageable by hand. Two harnesses plus an editor assistant, plus a code-review bot, plus whatever a new contributor brings, is not — and the parts that are not bridgeable by an import, such as MCP server definitions and subagent files, multiply the problem because each tool wants the same information in a different file format.

The most mature tool aimed at exactly this problem is **Ruler** (`github.com/intellectronica/ruler`, published to npm as `@intellectronica/ruler`). As of September 2026, it had roughly 2,900 GitHub stars and 160 forks, over 1,000 commits, an MIT licence, and real continuous integration. That combination matters more than the star count: this is a category where half-finished scripts are common, and Ruler is the one with evidence of sustained maintenance.

The model is generate-from-source rather than symlink-everything. You keep a `.ruler/`

directory as the single source of truth, primarily `.ruler/AGENTS.md`, plus any additional `.md` files which are concatenated in sorted filename order. Running `ruler apply` writes each target tool's native format: a root `AGENTS.md`, Claude Code's files under `.claude/`, Codex's `.codex/config.toml`, and the equivalents for more than thirty other tools including Cursor, Windsurf, Aider, Goose, Zed, and Gemini CLI. MCP server definitions are distributed too, configured through `ruler.toml`, which is the part that hand-maintenance gets wrong most often because the JSON and TOML shapes are genuinely different rather than merely differently named.

Two newer features are worth knowing about and not worth depending on yet. Subagent propagation takes `.ruler/agents/` and writes each tool's native subagent location; skills distribution takes `.ruler/skills/` and writes each tool's native skills directory. Both are explicitly experimental and off-by-default. Given that Claude Code and Codex subagents use incompatible file formats — Markdown with YAML frontmatter against TOML — a generator is the only plausible route to sharing them, but treat the output as something to review rather than trust.

The discipline this imposes is the thing to understand before adopting it. Generated files are added to `.gitignore` and rewritten from scratch on every `ruler apply`. An edit made directly to a generated `CLAUDE.md` or `.codex/config.toml` is silently discarded the next time anyone runs the command, with no conflict and no warning. If you adopt Ruler, the rule "never edit a generated file" has to be real, enforced in review, and understood by every contributor who might reach for the file they can see rather than the one that produces it.

Smaller alternatives exist and are not equivalent. `ai-rules-sync` takes a different approach, using live symlinks to a cached canonical copy instead of a generate step, which avoids the clobbering problem at the cost of depending on symlinks; it has roughly 37 stars. `RuleSync` covers similar ground. `AgentsMesh` adds drift checking in continuous integration via `agentsmesh check`, which is a genuinely useful idea if your concern is a contributor committing a hand-edited generated file. None of the three has Ruler's maintenance record, and presenting them as four comparable options would misrepresent the state of the category.

Note what no tool in this category attempts. None of them bridge permissions or sandbox configuration, because the two models are structurally different rather than differently spelled — Claude Code matches patterns against tool calls at the application layer, Codex sets an approval policy and an operating-system sandbox mode. That duplication is accepted and unavoidable, and the chapter on feature translation explains why.

No tool provides cross-harness semantic coordination. Codex and Claude Code now

provide worktree-based isolation in their own workflows, and ordinary Git worktrees can isolate simultaneous checkouts, but no coordination layer resolves competing design assumptions or shared external state. A community guide on running the two harnesses together (`github.com/shakacode/claude-code-commands-skills-agents`, in `docs/claude-code-with-codex.md`) reaches the same conclusion and recommends git worktrees: give each agent its own worktree on its own branch, and let git handle the merge the way it handles any other two-author change. That guide's wider advice matches everything in this chapter — universal instructions in `AGENTS.md`, `CLAUDE.md` as an `@AGENTS.md` import plus Claude-specific extras, and `~/.codex/config.toml` left deliberately tool-specific.

4.7 Codex Configuration Is a Supplement, Not the Project Contract

Codex may have user-level and repository-level configuration for model choice, approval behaviour, sandboxing, or instruction-file fallback. Those settings are more volatile than repository prose and may differ by client. Keep them small, documented, and safe to ignore when a user runs another interface.

At the time of drafting, this repository sets `project_doc_fallback_filenames = ["CLAUDE.md"]`, whose semantics are covered earlier in this chapter. The intent is narrow: if the normal Codex-facing file is absent, the existing Claude Code instruction file remains discoverable. It does not turn `CLAUDE.md` into a universal Codex configuration format, and it does not add `CLAUDE.md` as a second project instruction source when `AGENTS.md` is present. The current configuration reference also says that project-local `.codex/` layers are skipped for untrusted projects, so trust status is part of the setup rather than an implementation detail.

Do not put model preferences, approval choices, or secrets into shared project instructions unless every contributor should inherit them. A developer who needs a more permissive sandbox for a local experiment should be able to make that choice locally without changing the repository's review policy for everyone else.

4.8 A Practical Instruction Audit

When adding Codex to an existing Claude Code repository, audit the instruction system in this order:

1. Identify the current canonical project instructions and read them before writing an adapter.
2. Separate repository policy from Claude-specific features.
3. Add the smallest Codex-facing entry point that can be tested.

4. Check for contradictions between `CLAUDE.md`, `AGENTS.md`, local files, and user-level configuration.
5. Ask each harness, in a read-only session, which instructions it found and which rules it believes apply.
6. Run one bounded write task and inspect whether the agent followed the shared build, scope, and validation rules.

The audit is complete when both agents can answer the same repository questions and still retain their own harness-specific workflows. You do not need identical prompts or identical feature sets. You do need one understandable source of truth for what the repository considers a correct change.

5 Workflow Differences

If you already use Claude Code, the first Codex session can feel familiar enough to be misleading. You still describe a goal, let an agent inspect a repository, approve or reject actions, review a diff, and run tests. The important differences are in the defaults and in the places where the harness decides how to gather context, when to ask for permission, how much reasoning to spend, and how to continue after the first pass.

Treat Codex as a second harness rather than as a different spelling of the same workflow. The transferable skill is the engineering loop: establish the task boundary, inspect the relevant system, make a small change, verify it, and review the result. The parts you need to recalibrate are the hand-off between those stages and the amount of context you make explicit.

5.1 Exploration: Establish the Map Before Choosing a Route

Claude Code users often begin with a natural-language request and allow the agent to discover the repository while working. That can work well when the repository instructions, naming conventions, and test layout are already familiar. With a second harness, exploration has another purpose: it tests whether the new harness has found the same sources of truth that you rely on in Claude Code.

Start a Codex task with a read-only reconnaissance pass when the repository is unfamiliar or the change crosses several components. Ask it to identify the relevant instruction files, entry points, tests, build commands, generated files, and local conventions. Ask for paths and evidence, not just a summary. For example:

```
Inspect this repository without modifying files. Identify:
```

```
1. The project instruction files and which ones apply to this directory.
2. The likely entry points for the authentication flow.
3. The tests that exercise the flow.
4. The commands used for formatting, testing, and building.
5. Any generated or vendored files that should not be edited directly.

Give me a short map with file paths and explain what you would inspect next.
```

This prompt is useful when switching harnesses because it exposes instruction-discovery drift early. If Codex names a different build command, misses a nested instruction file, or treats a generated file as a source file, fix the context before asking it to implement anything. The same repository can produce different outcomes when the agents receive different local rules.

Exploration should be proportional to the task. For a one-line change in a well-known file, asking for a repository census creates noise and consumes context. For a migration or bug with several possible causes, skipping reconnaissance makes the first edit a guess. A good rule is to explore until you can state the change surface: the files that probably change, the tests that should fail or pass, and the commands that can prove the result.

Do not ask the agent to read every document "for context." Large instruction files, generated outputs, fixtures, and historical notes can crowd out the code that matters. Point it towards the relevant paths and ask it to follow references only when they affect the task. This is especially important when Claude Code and Codex use different instruction-file conventions: a thin compatibility file is useful only if the agent actually follows the path to the canonical guidance.

5.2 Planning: Make the Change Boundary Explicit

Planning has two meanings in agentic coding. It can mean a lightweight statement of intended edits before work begins, or it can mean a detailed implementation design with dependencies, alternatives, and test strategy. Both are useful, but they serve different tasks.

For a small change, ask for a compact plan and proceed quickly. For a risky change, require the plan to name invariants and verification steps. A useful plan answers four questions:

- What behaviour changes?
- What behaviour must remain unchanged?
- Which files or interfaces are in scope?
- How will you know the implementation is correct?

Claude Code users sometimes rely on a conversational plan that evolves as the agent explores. Codex can work that way too, but you get a more reliable hand-off if you distinguish discovery from commitment. Ask it to investigate first, then approve a plan once the evidence is visible:

```
First inspect the relevant code and tests. Do not edit files yet.
Then propose the smallest implementation plan. Include the expected files,
the tests you will add or update, and any uncertainty that could change the
plan. Wait for my approval before making changes.
```

This is not bureaucracy. It prevents a common failure mode in which the agent turns an ambiguous request into a broad refactor before it has located the existing abstraction. It also makes it easier to compare Claude Code and Codex: give both harnesses the same task boundary and inspect where their proposed approaches diverge.

Plans should not become a substitute for verification. A polished plan can still rest on a false assumption about the code. Once implementation starts, let the agent revise the plan when tests or source inspection contradict it. The useful distinction is between a plan that records current intent and a rigid script that prevents learning.

5.3 Implementation: Constrain Scope, Preserve Local Idioms

The implementation stage is where familiar-looking behaviour hides the biggest practical difference. An agent can produce a plausible patch while using the wrong abstraction, editing generated output, or changing more files than the request warrants. The harness's ability to inspect and modify files matters, but so does the instruction you give it about scope.

State the boundary in terms the repository can enforce. "Fix the parser" is weaker than "change the parser under `src/config`, preserve the public error type, add regression coverage for malformed input, and do not change the serialisation format." Include explicit exclusions when they matter: no dependency upgrades, no unrelated formatting, no generated-file edits, no API renaming.

Ask the agent to implement in a coherent slice rather than narrate every keystroke. Excessive micro-management can make the task brittle: the agent spends its context repeating instructions instead of checking the code. The opposite failure is an unbounded request such as "clean this up," which gives the agent no principled stopping point. A good implementation prompt states the contract, the allowed surface, and the acceptance checks.

When Codex is working in a repository that Claude Code has already shaped, tell it which local pattern to follow. Point to an existing analogous implementation instead of describing the style abstractly:

```
Implement this using the same boundary and error-handling pattern as
`src/importers/json_importer.py`. Keep the public API unchanged. Add tests
next to the existing importer tests. Do not edit generated files or update
dependencies. Stop after the focused tests pass and report the full test
command you ran.
```

The example does more than prescribe style. It gives the agent a nearby source of truth and a

stopping condition. If the two harnesses have different defaults for formatting, file discovery, or command approval, those explicit constraints reduce accidental divergence.

5.4 Testing: Treat Verification as Part of the Task

A test command is not a magic stamp of correctness. It is evidence about a particular slice of behaviour under a particular environment. Ask the agent to choose tests that correspond to the changed contract, then widen verification when the change crosses a boundary.

For a focused bug fix, the useful sequence is usually: reproduce or write the failing case, make the smallest change, run the focused test, run the relevant package or subsystem tests, and run broader checks if the risk justifies them. For a documentation or configuration change, validation may mean rendering, linting, schema checking, or inspecting the generated artefact rather than running a unit-test suite.

Tell Codex what "done" means before it edits. Otherwise an agent may stop after a syntactically valid patch or report that a test could not run without distinguishing an environmental failure from a product failure. Ask it to report the command, result, and any limitation:

```
Verify the change in layers:

1. Add or update a regression test for the reported behaviour.
2. Run the focused test file.
3. Run the relevant package test target.
4. If a check cannot run, report the exact command, error, and whether the
   limitation is environmental or caused by the patch.

Do not claim the task is complete based only on static inspection.
```

Claude Code and Codex may expose different approval or sandbox behaviour around commands, especially when a test needs network access, containers, or writes outside the immediate project directory. Keep the conceptual distinction clear: approval determines whether the agent may take an action; the sandbox determines what the execution environment allows. A command can be approved and still fail because the sandbox blocks it, or it can be permitted by the environment while still requiring a deliberate approval decision.

Testing is also a useful way to compare agents without relying on impressions. Give each harness the same repository state and acceptance criteria. Compare the tests each one chooses, the failures it notices, and whether its final explanation accurately describes what ran. "It felt more capable" is weak evidence; a reproducible verification record is stronger.

5.5 Review: Review the Diff, Not the Confidence

An agent's final summary is a useful index, not a substitute for review. The summary can be concise and accurate while omitting a subtle behaviour change, or it can sound confident

when a test was skipped. Review the actual diff and the evidence behind the claims.

Ask Codex for a review pass after implementation, including a fresh look at scope and failure modes:

```
Review the current diff as if you did not write it. Check:

- whether every changed file is necessary;
- whether the implementation preserves the existing public behaviour;
- whether the tests cover the failure mode rather than only the happy path;
- whether error handling, input validation, and compatibility were changed;
- whether generated files or unrelated formatting slipped in.

Report findings by severity, then list the verification commands and results.
Do not modify files during this review.
```

A second agent can be particularly useful as a reviewer because it approaches the diff without the original implementation narrative. That does not make it automatically independent: both agents can inherit the same misleading issue description, test gap, or repository instruction. If the change is high impact, review the behaviour from outside the agent loop as well. Run the relevant tests yourself, inspect interfaces, and check the final diff against the original request.

Use a clean review context when the task is complex. Continuing in the same session preserves useful context, but it also preserves the assumptions that produced the patch. A fresh session or a second harness can expose a missed edge case precisely because it has to reconstruct the reasoning from the repository and diff.

5.6 Sessions, History, Compaction, and Hand-off

"Continue" is not one operation. It can mean continue in the same conversation, resume a saved Codex chat, recover after a command failure, continue after context compaction, or hand work to a different harness. These mechanisms have different storage, context, and failure boundaries.

Layer	What persists	What it does not mean
Codex CLI history	Saved chats and local session metadata; the official CLI exposes `codex resume` and `/resume`	A Claude Code transcript or a repository checkpoint

Layer	What persists	What it does not mean
Local Codex storage	On the installation checked for this edition, this may include JSONL history/index files and SQLite thread/log databases under `$CODEX_HOME`	A stable public export format; exact names, schema, retention, and search behaviour may change
ChatGPT Codex chats	The Help Center documents account-retained chats for the ChatGPT app	The same retention or deletion rules for local CLI data or API state
Responses/Conversations API	Server-side conversation state, chained with `previous_response_id` or a durable Conversations API identifier	Codex CLI history, or state that Claude Code can read automatically

The current Codex CLI documentation describes `codex resume` as a way to reopen a recent chat or search older local chats. The developer command reference documents `codex resume --last` for the most recent chat in the current working directory and `--all` for sessions beyond that directory. A local Codex installation therefore has persistent application state. Treat `$CODEX_HOME` as an implementation boundary, not as a portable transcript store: do not copy its databases into a repository or make another agent depend on their filenames or schema.

This is separate from API conversation state. The OpenAI API conversation-state guide describes manual chaining with `previous_response_id` and durable Conversations API objects that can be reused across sessions, devices, or jobs. That is API state, not proof that a Codex CLI session is available to Claude Code. The ChatGPT Codex chat guidance also has account-specific retention and deletion rules; do not generalise them to every Codex surface.

Context compaction is different again. At the API level, the OpenAI compaction guide describes a compressed, opaque item that carries task-relevant prior state into the next context window. API compaction keeps an active task moving; it is not a delete operation on the saved conversation. Its summary can still omit an exact command, path, test result, or caveat, so verify important facts against the working tree and restate unresolved work after compaction. The exact automatic compaction behaviour is interface- and release-specific; do not infer the CLI experience from the API endpoint.

When resuming, inspect `git status`, the diff, generated files, and the last relevant test result before acting. A resumed chat restores conversational context, not a guaranteed repository state. If another process, agent, branch, or build changed the tree, the files and tests outrank the old conversation.

Codex and Claude Code do not share conversational state by default. A reliable cross-harness hand-off therefore contains two parts: the repository state and a short explicit note. The repository state is the source, diff, tests, and build outputs. The note records what is complete, what failed, the remaining contract, and the next allowed action:

```
The previous agent implemented the change in the current working tree.
Please inspect the diff before doing anything else. The intended contract is
[contract]. The focused test passed, but the broader suite was not run because
[reason]. Your job is to review the implementation, identify missing coverage,
and make only necessary corrections.
```

This hand-off works even when the next agent cannot open the first agent's session. It also prevents the common mistake of treating a conversational claim that "the task is finished" as evidence about the files.

Within one session, continuation is efficient when the agent has already inspected the right files and the next step follows directly. State a checkpoint explicitly: what is complete, what failed, and what must happen next. Keep durable facts in repository files only when the project benefits from that record; otherwise put the essential facts in the next prompt. A compact checkpoint should name the current diff, the last successful command, the unresolved failure, and the next allowed action.

5.7 Git: Use the Working Tree as the Contract

Git is the most reliable shared language between Claude Code and Codex. Both agents can inspect a branch, diff, status, history, and tests, but you should not assume that either one will choose the same commit or worktree strategy by default.

Before implementation, establish the starting point with `git status` and inspect any existing changes. If the working tree is already dirty, tell the agent which changes belong to the current task and which must remain untouched. "Do not overwrite my work" is useful, but naming the paths is safer. Ask for a diff review before staging or committing, especially when generated files are committed in the project.

A practical boundary looks like this:

```
Work only on the requested change. Preserve all pre-existing modifications.
Before editing, inspect `git status` and the existing diff. At the end, show
the files changed and the relevant diff summary. Do not commit, reset, rebase,
or discard changes unless I explicitly ask you to.
```

The exact commands and defaults for commits, branches, worktrees, and remote operations are product-specific and can change. Treat them as configuration and workflow choices, not as inherent properties of the model. Check the current Codex CLI documentation before relying on a particular workflow.

For parallel work, isolated worktrees or branches reduce accidental interference. They do not eliminate merge risk: two agents can make incompatible assumptions even when they edit different files. Keep parallel tasks partitioned by ownership and contract, then run an integration test and review the combined diff. If two agents must touch the same interface, serialise the design decision before parallelising implementation.

Codex's current CLI and desktop workflows can create a per-chat worktree through `/worktree` or `--worktree`, and can hand a finished worktree back to the local checkout for review. Claude Code can likewise request worktree isolation for a subagent. These conveniences change how the checkout is created, not the integration rule: one Git branch cannot be checked out in two worktrees at once, and detached or generated worktree names still need to be recorded in the hand-off. If ignored local files are required, use the worktree's documented include mechanism, such as `.worktreeinclude`, or a reproducible setup script; do not assume that `node_modules`, virtual environments, `.env` files, or local build caches follow the worktree.

5.8 Parallel Work: Divide by Contract, Not by File Count

Parallel agents are useful when the work has separable outcomes. "One agent per file" is a poor partition if all files implement one coupled behaviour. Better divisions are "research the existing API," "add focused regression coverage," "implement the backend adapter," and "review the resulting diff," provided each role has a clear output and a defined boundary.

Give each parallel worker the same repository rules and a narrow task contract. Tell workers whether they may edit files, whether they should commit, and how they should report conflicts. Keep a single owner for integration. The owner should inspect every branch or worktree, resolve interface differences, run the full relevant verification, and decide which changes belong in the final patch.

Parallelism increases throughput only when coordination costs stay below the time saved. It is a poor fit for exploratory debugging where each new observation changes the next step, or for a small task whose review cost exceeds its implementation cost. It also increases the chance of duplicated edits, inconsistent terminology, and tests that pass in isolation but fail together.

5.9 A Practical Cross-Harness Loop

For most repository work, the following loop transfers cleanly from Claude Code to Codex:

1. Establish the working-tree boundary and read the applicable instructions.
2. Explore until you can name the change surface and verification path.
3. Ask for a plan when the task has meaningful ambiguity or risk.
4. Implement the smallest coherent change using local patterns.
5. Run focused checks, then widen verification according to the change surface.
6. Review the diff and the evidence, ideally from a fresh context for risky work.
7. Record a checkpoint before handing the work to another session or harness.

The difference is not that one harness follows this loop and the other does not. The difference is how much of the loop each harness performs implicitly and how much you need to make explicit. If Codex is producing broad or surprising patches, tighten the task contract and require an inspection pass. If it is spending too much time describing obvious steps, shorten the plan and move to a focused test. Adjust the interaction pattern before concluding that the underlying model cannot handle the task.

6 Models and Reasoning Effort

Claude Code users often ask, "Which Codex model should I use?" That question is incomplete. The useful choice is a combination of model capability, reasoning effort, context available to the task, execution environment, latency, and cost. Exact model names and entitlements change; the task characteristics are more stable.

Think of model selection as choosing an engine for a job, and reasoning effort as deciding how long the engine should spend checking the route. A stronger model at a lower effort may be faster and more useful than a weaker model at maximum effort. A high-effort run can still produce a wrong answer if the repository context or acceptance criteria are wrong. Model controls improve the search; they do not replace a good task boundary and tests.

6.1 What the Controls Mean

The model is the underlying system generating decisions, tool calls, and text. Models differ in coding ability, instruction following, context handling, speed, availability, and cost. Those differences are not perfectly captured by a single intelligence ranking. A model that is excellent at a self-contained algorithm may be less useful for a large repository if it is slow to navigate or weak at maintaining constraints across many files.

Reasoning effort is a control over how much internal computation or deliberation the system allocates before producing an answer or taking the next action. The labels and exact behaviour depend on the product surface. Conceptually, lower effort favours speed and throughput; higher effort gives difficult tasks more room for decomposition, checking, and alternative consideration. It does not guarantee correctness, and increasing it can make a poor prompt more expensive without making it more precise.

Reasoning controls are surface- and model-specific. The current Codex CLI guide describes Low, Medium, High, Extra High, Max, and Ultra; its documentation defines Ultra as a mode that can use automatic task delegation. The Astra API page lists `low`, `medium`, `high`, `xhigh`, and `max` for `reasoning.effort`. Do not assume that CLI "Extra High" is a universal alias for API `xhigh`, or describe Ultra as merely another point on one effort slider. In every surface, higher effort can improve difficult work but takes longer and uses more tokens.

Context is the third control people often overlook. The model can only reason over the instructions, conversation, files, tool results, and other material available in the current context. More context is not automatically better: irrelevant logs and giant generated files can bury the contract and the code that matters. Curate context by pointing to the relevant paths, asking for targeted inspection, and preserving durable decisions in the repository when appropriate.

Execution capability is separate again. A model may be able to propose a correct command while the harness lacks permission, network access, credentials, or a suitable runtime to execute it. Keep model quality, reasoning effort, context quality, and execution permissions as separate variables when diagnosing a failure.

6.2 A Dated Snapshot of the Current Codex Models

The official Codex model guidance now recommends GPT-6.1 Sol for complex coding and GPT-6 Luna for focused work, with availability depending on the account and client. The GPT-6 Astra model documentation describes Astra as OpenAI's most capable model for complex reasoning, coding, computer use, research, and document creation. The following is a snapshot checked on 5 November 2026 against official documentation; it is not a promise that every model is available to every account, interface, or authentication method. GPT-6.1 Sol's launch includes eligible paid plans; Enterprise and Edu administrators must enable it, and Free and Go are not included at launch.

Model	Practical orientation	Good starting use
`gpt-6-astra` (Astra)	Highest-capability model for complex work across code, apps, and research	Hardest end-to-end tasks that need sustained reasoning and judgement
`gpt-6.1-sol` (6.1 Sol)	Recommended for complex coding and agentic workflows; OpenAI describes near-Astra performance at lower cost	Ordinary implementation through repeated, long-running work; evaluate against Astra on hard tasks
`gpt-6-luna` (6 Luna)	Efficient model for focused, high-volume tasks	Clear transformations, extraction, summaries, and bounded coding
`gpt-6-sol` (6 Sol)	Earlier GPT-6 Sol; named replacement in the GPT-5.5 retirement notice	Migration where the account offers 6 Sol but not 6.1 Sol
`gpt-6.1-terra`	Fast, efficient model for writing, visual understanding, and computer use	Clear explanations, visual inspection, and ordinary computer-use tasks; availability varies by surface
`gpt-5.5`	Retires from ChatGPT-authenticated Codex on 14 October 2026; API availability is separate	Migrate saved settings, scripts, custom agents, and scheduled tasks before retirement

GPT-5.4 and GPT-5.4 Mini retired from Codex with ChatGPT sign-in on 31 August 2026. That specific retirement does not affect the OpenAI API or Codex authenticated with an API key. GPT-5.3 Codex Spark retired on 14 September 2026 from the desktop app, CLI, and IDE extension; it is no longer a preview option. GPT-5.2 and GPT-5.3 Codex are also deprecated with ChatGPT sign-in. Model names, availability, retirement dates, and migration advice belong in a maintained reference section, not in permanent repository instructions.

The practical selection rule is simpler than the catalogue. Use GPT-6 Luna for bounded work and GPT-6.1 Sol for implementation, debugging, and complex agent work when available. Keep Astra for the most demanding tasks. Choose effort separately: the product guide suggests High for Luna and Light (`low`) for Astra; start with the client's default for 6.1 Sol. Re-evaluate on familiar tasks rather than carrying an old generation's effort setting across

unchanged. Luna supports Max but not Ultra; Ultra uses subagents, so it is also a workflow choice rather than just more reasoning time.

6.3 Choose by Task Shape

Use the least expensive configuration that can reliably satisfy the acceptance criteria, then increase capability when evidence says the task needs it. The goal is not to use the strongest available model for every prompt. The goal is to match the configuration to the uncertainty and cost of being wrong.

Task shape	Starting configuration	Increase effort or capability when…
Mechanical edit with clear tests	Fast or lower-effort option	The change crosses an abstraction boundary or tests are weak
Familiar bug with a reproducible failure	General coding option at moderate effort	The cause is non-local, intermittent, or concurrent
Unfamiliar repository exploration	Stronger context handling at moderate effort	The architecture is ambiguous or the first map conflicts with the code
Multi-file refactor or API migration	Stronger model with deliberate planning	Compatibility constraints, generated code, or many callers are involved
Security-sensitive, data-sensitive, or high-impact change	Stronger model at higher effort plus independent review	The threat model is unclear or verification cannot cover the risk
Review of an existing patch	A configuration different from the implementer's when possible	The reviewer repeats the same assumptions or lacks enough context

This table is a starting heuristic, not a benchmark. A "simple" change in a mature repository can be harder than a large but well-specified migration. The cost of failure matters as much as the amount of code: a small permission change deserves more scrutiny than a large comment update.

6.4 Exploration and Planning Usually Need Less Than Implementation of Ambiguity

High reasoning effort is most useful when the agent must resolve competing explanations, preserve many constraints, or make a design choice with expensive consequences. It is less useful when the task is already fully specified and the remaining work is mechanical.

For exploration, begin with enough capability to build a trustworthy map. Ask for evidence and paths. If the agent cannot distinguish the relevant subsystem from neighbouring code, increase capability or narrow the search before asking for a larger answer. Spending maximum effort on an unbounded repository tour often produces a long but shallow inventory.

For planning, effort should track the number of plausible designs and the cost of choosing the wrong one. A small bug with one obvious fix does not need a design essay. A compatibility migration, concurrency change, or schema transition does. Ask the agent to state assumptions and alternatives so that you can see whether extra reasoning is addressing real uncertainty.

For implementation, increase effort when the patch must preserve invariants across files, reason about edge cases, or modify code whose behaviour is not fully captured by tests. Keep the task bounded. A powerful model with a broad mandate can still turn a focused fix into an unnecessary redesign.

For testing and review, capability matters because the agent must interpret failures and look for omissions rather than merely produce code. Independent review is often more valuable than repeatedly increasing the implementer's effort. A second pass with a fresh context can challenge assumptions that a longer first pass has merely reinforced.

6.5 A Model-Selection Workflow

Instead of memorising a product matrix, use a short escalation loop:

1. Classify the task as mechanical, diagnostic, design-heavy, or high-impact.
2. Choose a fast or general configuration that can inspect the relevant code.
3. Set a clear acceptance test and ask for a bounded first pass.
4. Inspect the result and the evidence, not just the prose.
5. Increase reasoning effort or switch models only when the failure reveals a capability problem.
6. Re-run the same acceptance checks after changing the configuration.

This gives you a meaningful signal. If the first run failed because it never found the right file, a stronger model may not help until you improve the prompt or instruction layer. If it

found the right code but repeatedly misses a cross-file invariant, more reasoning or a stronger model may be justified. If it produced the right patch but could not run tests, the problem is execution configuration rather than model choice.

You can make the escalation explicit in a prompt:

```
Start with a focused investigation and do not edit yet. If the cause is
clear, propose the smallest fix. If there are multiple plausible causes,
compare them using the existing tests and call sites. Do not switch to a
broader redesign without explaining why the local fix would be unsafe.
```

The prompt helps the agent spend effort on uncertainty rather than on ceremony. You can then raise the effort setting for the same task if the investigation remains inconclusive, without changing the task contract.

6.6 Worked Example: Escalating a Repository Task

Consider the request: "update the primer build so the PDF uses the repository's pinned container, regenerate the committed outputs, and run the relevant checks." A fast, lower-effort setting is a reasonable first choice when the `Justfile` already names the runtime and the expected files are clear. The task contract should still name the source, generated outputs, and build command. The evidence for a successful first pass is a small source diff, regenerated HTML and PDF, and checks that inspect the outputs rather than only their timestamps.

Now suppose the first run finds two competing recipes, one figure generator that runs on the host, and a failed PDF check whose error does not identify the source of the problem. This is a visible escalation signal: the implementation crosses several files, the repository invariant is unclear, and the first verification path has contradicted the plan. Move to a stronger model or higher reasoning effort, retain the same task contract, and ask it to compare the recipes and trace the failure before editing. Record the decision:

Stage	Configuration	Evidence	Decision
First pass	Faster setting	Found the source recipe but missed a host-side figure path	Escalate
Second pass	Stronger setting with deliberate reasoning	Identified the bypass, repaired the recipe, and passed the focused build	Keep this configuration for the task

The model setting changed because the repository evidence showed unresolved cross-file reasoning. The scope, safety boundary, and acceptance checks did not change. If the command

had been blocked by a missing container runtime, changing the model would have been the wrong response; the execution environment would need fixing instead.

6.7 Fast, Strong, and Specialised Are Not Permanent Categories

Product documentation may describe models using categories such as fast, general-purpose, reasoning-focused, coding-focused, or preview. Treat those categories as useful orientation, not permanent technical properties. A model can be renamed, retired, reclassified, made available in a different interface, or exposed with different controls. The same label may also behave differently across a chat product, command-line tool, application programming interface (API), or hosted execution environment.

For that reason, this primer keeps exact model names, prices, quotas, rate limits, context sizes, and plan entitlements in a dated snapshot rather than in its core workflow advice. Keep the durable guidance task-oriented, and recheck the official model and usage documentation before relying on any volatile detail.

Do not infer that a model available in one Codex surface is available in every other surface. Availability can depend on account, organisation, region, product, authentication method, rollout stage, or API access. The final version should link to official documentation for these points and state the date checked.

6.8 Cost and Latency Are Part of Engineering Quality

The cheapest run is not always the cheapest workflow. A low-cost configuration that creates three rounds of repair, review, and failed testing can cost more time and compute than one deliberate first pass. Conversely, using maximum effort for every search and formatting change wastes budget and slows feedback.

Estimate the cost of being wrong. For a reversible documentation edit, optimise for fast iteration. For a production migration, optimise for evidence and review. For an exploratory question, use a bounded investigation before committing to a long autonomous run. Track elapsed time, number of repair cycles, verification quality, and the final diff, not just the nominal model price.

When comparing Claude Code and Codex, keep the comparison fair. Use equivalent task descriptions, the same repository state, comparable permissions, and the same acceptance checks. Record whether a run needed human correction and whether the reported test results were accurate. Exact prices and quotas should be captured separately because they are volatile and can differ by product surface. Confirm which cost and usage metrics are exposed

by the reader's chosen Codex interface before adding a comparison example.

6.9 A Compact Selection Rule

Start fast when the task is bounded and mechanically verifiable. Move to a stronger model or higher reasoning effort when the task is ambiguous, cross-cutting, or expensive to get wrong. Improve the context and acceptance criteria before escalating when the failure is about repository discovery or execution. Add an independent review when the risk remains high after the implementation looks correct.

That rule keeps model selection subordinate to engineering discipline. The model is one component of the loop; the harness, instruction files, permissions, tests, and review process determine whether a plausible answer becomes a safe change.

6.10 Verification Notes for the Final Edition

The concepts in these sections are intended to remain stable, but the final primer should verify and date-stamp any product-specific claims about:

- exact Codex model names, aliases, and model families;
- which models and reasoning controls are available in each interface;
- the meaning and range of reasoning-effort settings;
- context limits, rate limits, quotas, and usage accounting;
- prices, subscription terms, API billing, and plan entitlements;
- support for commits, branches, worktrees, remote operations, and parallel execution;
- current sandbox, approval, network, and container behaviour.

Use official OpenAI documentation for those checks. Do not turn the verification list into a permanent matrix unless someone owns updating it.

7 Feature Translation

If you already use Claude Code, the first Codex session can feel familiar enough to be misleading. Both tools can inspect a repository, edit files, run commands, use external tools, and work through a multi-step coding task. That does not make their controls interchangeable. A feature with the same informal name may have different discovery rules, different defaults, or a different boundary between the model and the harness.

The useful question is not "What is the Codex version of this Claude Code feature?" It is "What job was this feature doing, and which Codex mechanism, if any, does that job?"

Sometimes the answer is a direct analogue. Sometimes the answer is a partial analogue that needs a different workflow. Sometimes the safe answer is that no equivalent exists and you should use a repository, shell, or CI convention instead.

This section uses three labels:

- **Direct equivalent** means the same broad capability exists and can usually carry the same intent, although syntax and configuration may differ.
- **Partial equivalent** means the goal can be achieved, but the lifecycle, scope, or guarantees differ enough that a copy-and-paste migration is unsafe.
- **Non-equivalent** means there is no reliable one-to-one feature. Use a different layer of the system.

The labels describe the capability, not the product quality. A partial equivalent can be the better tool for a particular job. It requires a deliberate translation.

7.1 Start With the Job, Not the Feature Name

Claude Code users often carry a feature-shaped mental model into Codex. For example, they may ask, "Where do I put my hook?" before deciding what the hook was protecting. Was it formatting changed files? Blocking a dangerous command? Injecting context? Recording an audit event? Those are four different jobs and may belong in four different places.

A useful translation has four parts:

1. **Purpose:** what outcome did the Claude Code feature provide?
2. **Trigger:** what caused it to run: every prompt, a tool call, a lifecycle event, or an explicit request?
3. **Authority:** was it advisory, a gate that could block an action, or an action with its own side effects?
4. **Scope:** did it apply to one session, one repository, one user, or the whole organisation?

For example, a Claude Code hook that runs a formatter after an edit has a different translation from a hook that rejects access to production credentials. The first can often move to a formatter command, pre-commit check, or CI job. The second needs a security boundary that does not depend on an instruction to the model.

7.2 Worked Example: Translating a Claude Code Hook and Skill

Suppose a Claude Code project contains a `PostToolUse` hook that runs a Markdown linter after edits, and a `/primer-audit` skill that tells Claude how to inspect the resulting HTML

and PDF. They are related, but they do different jobs. The hook is deterministic feedback: it should run after the relevant edit without asking the model to remember it. The skill is procedural knowledge: it tells the agent how to interpret failures and what evidence to record.

Translate the behaviour in four steps:

1. Move the linter command to a repository-owned `just lint-md` recipe or CI check. That gives both harnesses the same executable check and makes the failure visible even when no agent is present.
2. Keep the portable audit procedure in `.agents/skills/primer-audit/SKILL.md` and expose a Claude Code wrapper under `.claude/skills/primer-audit/` only if Claude needs different invocation or tool names.
3. In Codex, run the same recipe first and then invoke the skill or give the agent the same audit contract: inspect source, map snippets to tests, render representative pages, and report unresolved evidence separately.
4. Test the failure boundary by introducing a harmless Markdown error in a disposable branch. The linter should fail without relying on the model to notice the error; the audit procedure should explain the failure and identify the file and command that produced it.

The original trigger, authority, and side effects are now explicit. The repository check owns deterministic enforcement, the shared skill owns reasoning about evidence, and the harness-specific wrapper owns only invocation. If Codex has no hook with the same event semantics, do not pretend that a prompt saying "always run the linter" is equivalent enforcement. Keep the command in `just` or CI and use the prompt as a reminder.

The table below is a starting map. It is intentionally about responsibilities rather than exact command names or file paths.

Claude Code concept	Codex-facing translation	Status	Main caution
Project instructions (`CLAUDE.md`)	`AGENTS.md`	Direct capability, different discovery	Recent Claude Code releases can read `AGENTS.md`, but the default selects `CLAUDE.md` over it when both exist.

Claude Code concept	Codex-facing translation	Status	Main caution
Project skills	Project skills	Direct; same specification	Claude uses `.claude/skills/<name>/SKILL.md`; Codex uses `.agents/skills/<name>/SKILL.md`. Skill text that names harness-specific mechanisms is still harness-specific.
Custom commands	Custom prompts	Partial, and shrinking	Claude uses `.claude/commands/*.md`; Codex's `~/.codex/prompts/*.md` is deprecated in favour of skills. Flat directory only, no subdirectories.
Subagents	Subagents	Direct in concept, non-portable in format	Claude uses `.claude/agents/*.md`; Codex uses `~/.codex/agents/*.toml` or `.codex/agents/*.toml`. Markdown with frontmatter and TOML have no official translation tool.

Claude Code concept	Codex-facing translation	Status	Main caution
Hooks	Hooks	Direct in concept, non-portable in format	Claude uses `.claude/settings.json`; Codex uses `hooks.json` or a `[hooks]` block in `config.toml`. Event names, payloads, and trust models differ.
MCP servers	MCP servers	Direct at the protocol level, non-equivalent at the configuration level	Claude uses `.mcp.json`; Codex uses `[mcp_servers.<name>]` tables in `config.toml`. There is no shared configuration file or official converter.
Permissions (`allow`/`deny`/`ask` rules)	`approval_policy` in `config.toml`	Partial	Claude Code matches patterns against tool calls; Codex decides how often to ask, not which commands match.

Claude Code concept	Codex-facing translation	Status	Main caution
Sandboxing	`sandbox_mode` plus `[sandbox_workspace_write]`	Non-equivalent scope	Both harnesses have sandboxing, but Codex exposes a broader OS-level filesystem/network policy in this configuration model, while Claude Code's Bash sandbox does not automatically constrain file tools, MCP servers, or hooks.
Enterprise policy	`requirements.toml` and `managed_config.toml`	Different policy models	Codex can centrally constrain sandbox and approval settings; Claude Code has managed settings that can require its Bash sandbox and control instructions and permissions.
Tool integrations	MCP, built-in tools, connected apps, or a repository-side adapter	Partial	The existence of a connector does not imply that the current session can use it or write through it.

Two patterns run through that table. Where the capability is described by an open specification — skills, and MCP at the protocol level — the two harnesses converge and content moves with little or no translation. Where the capability is configuration, every row is a separate file in a separate format, and the translation is manual. The convergence is real and

recent; the configuration fragmentation is not improving.

Product names, supported surfaces, configuration locations, approval modes, plugin behaviour, and the availability of delegated-agent features change more quickly than repository conventions. Check the current Codex documentation for the exact release and interface you are documenting. This chapter stays concept-first so those checks do not require a rewrite.

7.3 Skills: Reusable Workflow Knowledge

A skill is best understood as a reusable package of instructions, examples, and workflow rules. It is not the same thing as a model, a tool, or a permission grant. A skill can tell an agent how to perform a task; it cannot safely grant access to a production system merely by describing that access.

This is close to a direct equivalent when the Claude Code skill is mostly procedural guidance:

```
When asked to change a database schema:
1. Inspect the migration conventions in this repository.
2. Write a reversible migration.
3. Update the fixture and migration test.
4. Run the database-specific checks.
5. Report the migration and its rollback path.
```

The translation becomes partial when the Claude Code skill depends on harness-specific behaviour. Examples include a special invocation syntax, a particular subagent type, a lifecycle hook, or an assumed tool name. Preserve the workflow and rewrite the parts that refer to those mechanisms. The old command name alone is no reason to preserve it.

Keep three kinds of guidance separate:

- **Repository guidance:** conventions that should apply regardless of agent, such as the test command, generated-file policy, or source-of-truth rules. Put this in neutral project documentation or the appropriate instruction files.
- **Workflow guidance:** a reusable procedure for a class of tasks. This is a good candidate for a skill.
- **Harness guidance:** instructions about how a particular agent discovers context, asks for approval, or invokes tools. Keep this close to the harness-specific skill or adapter.

The practical migration pattern is to make the procedure agent-neutral first, then add a short Codex-specific wrapper. The wrapper should explain what the agent must inspect and what it must report, not duplicate the entire repository manual.

Current OpenAI guidance describes a skill as a `SKILL.md` file with optional scripts, references, and assets. Codex skills are available across the CLI, IDE extension, and ChatGPT desktop surfaces, while plugin-bundled skills can also appear in connected ChatGPT or Work

experiences. The exact discovery and invocation controls differ, and arbitrary API or SDK workflows are not automatically Codex skills. See the official skill-building documentation before documenting a directory name, manifest field, or invocation syntax.

7.3.1 Skills Are the Closest Thing to a Shared Extension Format

Skills are much closer to a shared extension format than hooks, permissions, or subagents, because both products use the open Agent Skills `SKILL.md` shape. That does not mean that a skill written for one harness is automatically discoverable or behaviourally identical in the other. The directory, optional frontmatter fields, invocation surface, tool names, and approval behaviour can still differ. Treat the skill body as portable procedure and the surrounding directory and adapter as integration work.

The shared design is progressive disclosure, but the frontmatter requirements are not identical. Codex requires a `name` and a `description`; Claude Code can derive a name from the directory and supports additional Claude-specific fields. Use explicit `name` and `description` fields for a cross-harness skill, and assume that only the shared subset will port cleanly. The initial name-and-description metadata is loaded before the full procedure: Codex's documentation puts a concrete budget on that preamble: the loaded name-and-description set should stay under about 2% of the context window, or roughly 8,000 characters. The body of `SKILL.md`, along with any bundled scripts, references, and assets, loads only when the skill is actually invoked. That is what makes it viable to have fifty skills installed: the standing cost is fifty short descriptions, and the full cost is paid one skill at a time.

The directories differ, and the Codex side is the more interesting of the two:

Scope	Claude Code	Codex CLI
Project	`.claude/skills/<name>/SKILL.md`	`.agents/skills/<name>/SKILL.md`, searched from cwd upward through parent directories to the repository root
User	`~/.claude/skills/`	`$HOME/.agents/skills/`
Administrator	Managed settings	`/etc/codex/skills`

Note the path. Codex does not use `.claude/skills`; it uses `.agents/skills`. The format is shared, but discovery is not. The most reliable team patterns are:

- keep a canonical skill source in a neutral directory and generate both `.claude/skills/` and `.agents/skills/` during a checked-in configuration update;

- use a symlink only when the repository's supported platforms and Git settings make symlinks reliable; or
- keep two thin wrappers with one shared procedure file when the two harnesses need different invocation or tool instructions.

Do not assume that a `.claude/skills` directory is visible to Codex, or that a `.agents/skills` directory is visible to Claude Code. Test discovery in both tools after installing or moving a skill. Also keep the initial `name` and `description` concise: Codex uses a bounded initial skills list and both tools load the full body only when a skill is selected.

The caveat is about content, not format. A skill that describes a procedure — how to write a reversible migration, how to review a rendering diff, what the project's definition of done is — moves between harnesses untouched. A skill whose body says "invoke the `render-primer` hook" or "delegate this to the `explorer` subagent" is naming a mechanism that may not exist, or may behave differently, on the other side. Write the procedure in terms of commands and files, which both harnesses can execute, and keep the harness-specific invocation in a short wrapper section that a reader can see is harness-specific. If a skill needs a Claude-only or Codex-only tool, say so in the description or wrapper rather than letting the agent discover the mismatch halfway through a task.

7.3.2 Custom Slash Commands Are Being Absorbed Into Skills

Claude Code users who rely on `.claude/commands/*.md` will look for the equivalent, and it exists: `~/.codex/prompts/*.md`, where the filename minus the `.md` extension becomes the command name, so `review-diff.md` gives you `/review-diff`. The directory is flat; subdirectories are not supported, which rules out the namespacing that a large Claude Code command collection tends to grow.

OpenAI's own documentation now marks this deprecated in favour of skills. Existing prompt files continue to work, so there is no urgency, but the direction is unambiguous, and it is the second piece of evidence that both vendors regard skills rather than slash commands as the durable unit of reusable workflow. If you are porting a Claude Code command collection, port it to skills rather than to prompts. The work is the same size and the destination is the one both harnesses are investing in.

7.4 Hooks: Automation Around the Agent

Claude Code hooks are often used for deterministic actions around agent events: checking a command, formatting a file, logging an action, or refusing a risky operation. Codex has a

real hooks system too, and it is closer to Claude Code's than anything else in this chapter apart from skills.

Codex loads hooks from four locations, and all of them may contribute handlers: `~/.codex/hooks.json`, an inline `[hooks]` block in `~/.codex/config.toml`, `<repo>/.codex/hooks.json`, and a `[hooks]` block in `<repo>/.codex/config.toml`. Matching command handlers may run concurrently; do not encode ordering dependencies between them. Splitting hooks across a JSON file and a TOML block in the same scope is legal and is a good way to confuse a future maintainer; pick one per scope.

The lifecycle events are `SessionStart`, `SessionEnd`, `PreToolUse`, `PostToolUse`, `PermissionRequest`, `PreCompact`, `PostCompact`, `UserPromptSubmit`, `SubagentStart`, `SubagentStop`, `Stop`, and `Interrupt`. A Claude Code user will recognise almost all of that, because the event set closely matches Claude Code's own. This is the third quiet convergence point in the chapter, and unlike skills it does not appear to come from a shared specification — it looks like two teams arriving at the same decomposition of an agent's lifecycle. The practical consequence is that porting the *logic* of a hook between the two harnesses is usually straightforward, even though porting the file is not possible at all.

`PreToolUse` is the event with authority. A Codex `PreToolUse` hook can deny an action outright, either by returning `permissionDecision: "deny"` or by exiting with status code 2, and it can rewrite the tool's input before the call proceeds. That is the same shape as Claude Code's `PreToolUse` hooks, including the exit-code-2 convention. `PostToolUse` can provide feedback after an action but cannot undo a command that already ran, which is the constraint that decides where a check belongs: anything that must prevent an outcome goes in `PreToolUse`, and anything in `PostToolUse` is reporting.

A minimal Codex hook in the TOML form looks like this:

```toml
[[hooks.PreToolUse]]
matcher = "^Bash$"

[[hooks.PreToolUse.hooks]]
type = "command"
command = "script_path"
timeout = 30
```

The `matcher` is a regular expression against the tool name, and the nested array lets one matcher run several commands in order. Compare that with Claude Code's hook configuration in `.claude/settings.json`, which is JSON with its own matcher conventions: the concepts line up, the text does not, and there is no converter.

Codex adds a trust gate that Claude Code does not have, and it is worth understanding before you write a hook that a colleague will inherit. A non-managed hook does not run

until a user has explicitly reviewed and trusted it, and the trust record is keyed to a hash of the hook itself. The `/hooks` command in the CLI is where that review happens. Editing the hook changes the hash and invalidates the trust, so a modified hook is re-reviewed rather than silently inheriting the old approval. This is a genuine safety property: a repository you clone cannot run arbitrary commands on your machine through its committed hook configuration just because you opened Codex in it.

Managed hooks, pushed through enterprise configuration, bypass that review by design — an organisation's administrators are the trust decision in that case. Administrators can also set `allow_managed_hooks_only = true`, which disables non-managed hooks entirely. If you are writing hooks for a team inside a managed Codex deployment, check that setting before investing in a repository hook that may never be permitted to run.

See the Codex hooks documentation for the current event contract and payload shapes.

Translate hooks by moving each responsibility to the narrowest deterministic layer:

Original hook responsibility	Safer translation target
Format changed source files	Formatter command, pre-commit hook, or CI check
Reject a known-dangerous command	Shell wrapper, restricted runner, container policy, or human approval
Add repository context	Instruction file, skill, or explicit prompt context
Notify a team after a change	CI automation or a separately reviewed integration
Record an audit event	Version-control, command-runner, or organisation-level logging

This matters because a model prompt is a poor substitute for a deterministic gate. "Do not run this command" in an instruction file can guide an agent, but it cannot protect against a separate shell, a developer mistake, a compromised dependency, or a different automation path. Put safety-critical controls below the agent.

If the target Codex interface does provide lifecycle hooks, first test a trivial hook that only records an event. Confirm when it runs, what input it receives, whether a non-zero exit blocks the action, and whether it runs in the same sandbox as the command it is observing. Only then move a formatter or policy check into it.

The event names overlap heavily, but do not assume that ordering, payload shape, or the exact blocking contract carries over unchanged. Port the logic of the hook, not the file, and verify the ported version with a trivial event-recording hook before you let it deny anything. If a required behaviour turns out not to be supported, use repository scripts, git hooks, continuous integration, or an explicit review step instead — those layers work regardless of which harness is driving.

7.5 MCP: Same Protocol, Different Connection

The Model Context Protocol (MCP) is a standard way for an AI application to discover and call tools or retrieve context from an external server. At that protocol level, an MCP server is a direct conceptual equivalent: the same server may be usable from both Claude Code and Codex. Current Codex host documentation describes local STDIO and Streamable HTTP connections, OAuth or bearer authentication, and CLI commands such as `codex mcp add`, `codex mcp list`, and `codex mcp login`. The ChatGPT desktop app, CLI, and IDE share the host configuration model, while hosted plugin tools are a separate surface.

The harness-level mapping is only partial. Each client decides how it stores server configuration, which transports it supports, how it handles authentication, whether it exposes every server capability, and how it asks for approval. A server that works in Claude Code may therefore require configuration or permission changes before Codex can use it.

Treat an MCP connection as three separate questions:

1. **Can the client discover the server?** Configuration and process startup must work.
2. **Can the session call the tool?** The tool may be visible but unavailable because of approval, account, network, or workspace restrictions.
3. **What may the tool do?** A connected tool can have read and write operations, and those operations may have external side effects.

A good first test is read-only and narrow:

```
Use the configured issue-tracker connection only to read issue ABC-123.
Do not add comments, change fields, transition the issue, or call any other tool.
Report whether the connection is available and quote only the issue title and status.
```

If that works, test a harmless write separately, with an explicit approval boundary. Do not infer write authority from a successful read. Do not place secrets in an instruction file or prompt when the connector has a supported authentication mechanism.

MCP also does not turn an external system into trusted truth. Validate identifiers, minimise the requested scope, and make the agent report what it actually read or changed. For a production system, prefer an integration with its own audit trail and permission model over

a long prompt that says the agent should be careful.

7.5.1 The Configuration Is Not Shared, and There Is No Converter

The protocol is common. The configuration file is not, and this is the place where a Claude Code user most often assumes a shared standard that does not exist. The two harnesses use different files, different formats, and different field names.

Claude Code reads `.mcp.json` at the project root. The file is JSON with a top-level `mcpServers` object, and each server entry carries a `type` field — `stdio`, `http` or `streamable-http`, `sse` (deprecated), or `ws` — plus `command`, `args`, and `env` for a local process, or `url` for a remote server. There is a documented gotcha here that costs people an afternoon: an entry with a `url` but no explicit `type` is read as `stdio`, which cannot work for a remote server, and the entry is skipped. The failure is a server that is not there, not an error message. Always set `type` explicitly.

Codex CLI reads TOML tables from `~/.codex/config.toml` for user scope, or `.codex/config.toml` for project scope. Project configuration is additive to the global file rather than replacing it, and — as with the instruction-file fallback discussed earlier — the project layer is only loaded for trusted projects. Servers are declared under `[mcp_servers.<name>]`:

```
[mcp_servers.context7]
command = "npx"
args = ["-y", "@upstash/context7-mcp"]
env_vars = ["LOCAL_TOKEN"]

[mcp_servers.context7.env]
MY_ENV_VAR = "MY_ENV_VALUE"

# remote/HTTP form
[mcp_servers.figma]
url = "https://mcp.figma.com/mcp"
bearer_token_env_var = "FIGMA_OAUTH_TOKEN"
http_headers = { "X-Figma-Region" = "us-east-1" }
```

Two details in that example have no Claude Code counterpart by the same name. `env_vars` is a list of variables to pass through from the ambient environment, which is distinct from the `[mcp_servers.<name>.env]` table that sets explicit values. And `bearer_token_env_var` names the variable holding a token rather than the token itself, which is the right pattern to copy regardless of harness: the credential stays in the environment and out of the committed file.

Codex also exposes operational fields that matter for a server that is slow or optional: `startup_timeout_sec` (default 10) and `tool_timeout_sec` (default 60), plus `enabled`, `required`, and `enabled_tools` / `disabled_tools` for trimming a chatty server's tool list

down to the handful you actually want in context. That last pair is worth using. A server that advertises forty tools consumes context in every session whether you call it or not.

There is no official converter between the two formats as of this research, and the mapping is not mechanical enough to make a five-line script safe — the transport field, the environment handling, and the timeout fields all differ in shape rather than in spelling. In practice, teams either hand-maintain both files, accepting that they will drift, or generate both from one source with a tool such as Ruler, which distributes MCP server definitions through its `ruler.toml`. If you hand-maintain, put the two files next to each other in review so a reviewer can see when one changed and the other did not.

7.5.2 Exposing One Harness to the Other as an MCP Server

There is a more experimental pattern worth knowing about, and it inverts the problem: instead of sharing configuration, you let one harness call the other as a tool. `codex mcp-server` runs Codex as a stdio MCP server, which means Claude Code can be configured to call it:

```
{
  "mcpServers": {
    "codex": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "codex", "mcp-server"]
    }
  }
}
```

That configuration appears in a real public repository, `ogmios2/claude-code-codex-mcp`, and the pattern is described in a small number of blog posts. Treat it as niche and experimental rather than mainstream: the sourcing is thin, the delegation semantics are not documented the way a first-party feature would be, and you now have two agent loops with their own approval and sandbox settings nested inside each other, which makes reasoning about authority considerably harder. It is genuinely interesting for a case where one harness has a tool integration or a model the other lacks. It is not a substitute for getting the shared instruction file right.

Confirm the current Codex surface's MCP support, supported transports, configuration method, authentication flow, and whether individual tools require confirmation. These details are volatile. The protocol relationship is stable; the client behaviour is not.

7.6 Subagents: Delegation Is Not Parallelism by Itself

Claude Code users may use subagents to give a bounded task to a separate context: inspect tests, review a design, search for callers, or implement a small independent change. This is an

area where the answer has changed: early 2025 Codex had nothing comparable, and current Codex has subagents as a generally available feature with its own configuration format, scoping rules, and concurrency controls. Simon Willison, reviewing the feature when it shipped, described it as "very similar to the Claude Code implementation." That is a fair assessment of the concept. It is not true of the files.

7.6.1 How Codex Subagents Are Defined

A Codex subagent is a TOML file. Personal definitions live in `~/.codex/agents/` and project definitions in `.codex/agents/`. Three fields are required — `name`, `description`, and `developer_instructions` — while optional fields describe the model, reasoning effort, and other agent metadata. The parent session's permissions, provider, and MCP servers remain the authority for the delegated work; a custom agent definition is not an independent security boundary.

The key safety lesson for a Claude Code user is the opposite of an assumed per-agent sandbox: writing "do not edit files" into a delegated prompt is not an operating-system guarantee. If a delegated task needs read-only or path-isolated execution, use a read-only parent session, a separate worktree, or a separate process with the required sandbox. Restrict MCP access at the parent/session or organisation layer and verify the effective configuration before the task starts.

Codex ships three built-in subagents — `default`, `worker`, and `explorer` — and a custom definition with one of those names overrides the built-in. That is convenient and also a trap: naming your own agent `explorer` silently replaces a built-in that other instructions or skills may reference.

Subagents can be invoked by your request or by the main agent acting on an instruction it read, whether that instruction came from `AGENTS.md`, a skill, or your prompt. Do not rely on description-only selection: applicable project or skill instructions may cause delegation, but the exact trigger depends on the interface and workflow. Make the delegation boundary explicit when it matters, and verify which agent actually ran.

Concurrency and defaults are set globally in an `[agents]` block:

```
[agents]
enabled = true
max_concurrent_threads_per_session = 4
default_subagent_model = "gpt-6-luna"
default_subagent_reasoning_effort = "high"
```

`max_concurrent_threads_per_session` is the control that stops an enthusiastic main agent from fanning out further than you can review. Set it deliberately. This example assigns

bounded tasks to Luna; use an available Sol or Astra model for a delegate whose task needs more capability. Check the subagent configuration and model availability before copying it.

7.6.2 The Formats Do Not Port, and One Rough Edge Is Unresolved

Claude Code subagents are Markdown files with YAML frontmatter in `.claude/agents/`, carrying fields such as `description`, `tools`, and `model`, with the body serving as the agent's instructions. Codex subagents are TOML, with the instructions in a `developer_instructions` string. The information overlaps almost completely; the serialisation shares nothing. No bridging or translation tool exists as of this research. Ruler's experimental `.ruler/agents/` propagation is the only attempt, and it is explicitly not solid yet.

One unresolved rough edge is worth flagging rather than glossing. GitHub issue `openai/codex#15250` reports that custom subagents placed in `.codex/agents` are not reliably reachable from tool-backed sessions, despite the documentation implying that they should be. Treat project-scoped Codex subagents as working but not yet dependable: verify that yours is actually reachable in the session type you intend to use it from, rather than assuming the documented scoping holds everywhere.

"Subagent" therefore still does not guarantee the same context isolation, tool access, or result handoff across the two harnesses, even though the concept now exists on both sides.

Translate a subagent task into an explicit contract:

```
Role: repository investigator
Scope: inspect only src/auth/ and its tests
Question: where is token expiry validated, and which tests cover it?
Restrictions: do not edit files, do not run commands that write files
Output: a short report with file paths, line numbers, and missing-test hypotheses
```

The contract makes the delegation useful even if it runs as a normal second session. It also prevents a common failure mode: asking several workers to modify overlapping files and then trying to merge their incompatible assumptions.

Parallelise only independent work. Good candidates are separate read-only investigations, test discovery, or analysis of disjoint modules. Keep architectural decisions, shared-file edits, migrations, and final integration under one coordinating context unless you have a deliberate merge strategy.

For every delegated task, define:

- the files or subsystem it may inspect;
- whether it may edit anything;
- the evidence it must return;
- how stale or contradictory findings should be reported; and

- who owns the final decision.

The exact Codex support for delegated agents, parallel workers, context sharing, model selection per worker, and result synthesis depends on the interface and current release. Do not promise a particular subagent command or assume that a delegated worker has broader authority than the parent session. Current Codex subagents inherit the parent session's permissions, provider, and MCP servers; a custom-agent definition supplies instructions and identity, not an independent OS-enforced sandbox. Use a separate session or worktree when you need stronger isolation, and verify the effective permissions before delegating a write task.

7.7 Permissions and Sandboxing Are Different Controls

This distinction is the one most likely to cause trouble for a Claude Code user. An approval or permission control is a decision about whether an action is authorised. A sandbox is a restriction on what the process can access or execute even if the agent tries. They overlap in purpose, but they are not substitutes.

Consider a command that deletes temporary files:

- An **approval** asks whether you, the policy, or the harness permits the agent to run it.
- A **sandbox** may prevent it from reaching files outside the working directory, or may restrict network access.
- A **repository rule** may tell the agent not to delete files at all.
- A **backup or version-control policy** may make recovery possible if the other layers fail.

Use all four layers where the consequence justifies them. Do not treat "the agent asked first" as proof that the command is safe, and do not treat "the process is sandboxed" as proof that the requested task is authorised.

The two harnesses do not merely use different names for the same control. They are structurally different mechanisms, and no shared convention or schema exists to bridge them. State that plainly to yourself before trying to port a policy, because the translation is genuinely lossy.

Claude Code expresses permissions as pattern rules in `.claude/settings.json`, scoped at user, project, and local level. Rules go into `allow`, `deny`, or `ask` lists and are matched against tool names and command patterns, and enforcement happens in the application: Claude Code consults the rules when it is about to make a tool call and acts accordingly. The granularity

is the pattern. You can permit `Bash(git status)` and refuse `Bash(git push:*)`, and the distinction is expressed entirely in text.

Codex splits the same territory across two orthogonal `config.toml` fields. `sandbox_mode` takes `"read-only"`, `"workspace-write"`, or `"danger-full-access"`, and `workspace-write` is tuned by a `[sandbox_workspace_write]` block carrying `writable_roots` and a `network_access` flag:

```
sandbox_mode = "workspace-write"
approval_policy = "on-request"

[sandbox_workspace_write]
writable_roots = ["/home/dev/project", "/home/dev/.cache/project"]
network_access = false
```

That is enforced by the operating system, not by the agent: macOS Seatbelt, the Linux/WSL2 sandbox using bubblewrap, and a native Windows sandbox. The current sandbox guide recommends installing the distribution's `bubblewrap` package on Linux or WSL2; its bundled fallback still needs unprivileged user namespaces. A command that violates the sandbox fails the way any sandboxed process fails, regardless of what the model intended or what any instruction file said.

`approval_policy` is the second, independent field, and it controls how often Codex pauses to ask rather than what the sandbox permits. `"never"` and `"on-request"` are the simple values, and a more granular form exists:

```
approval_policy = { granular = { sandbox_approval = true, rules = true, mcp_elicitations = true, request_permiss
```

These booleans control whether each category of approval prompt can appear; they do not grant the requested action. The configuration reference explicitly rejects `approval_policy = "untrusted"` and deprecates `"on-failure"`. Use `"on-request"`, `"never"`, or the supported granular form. Project `trust_level = "untrusted"` remains valid and is a different setting.

The orthogonality is the part worth internalising. A session can be highly permissive about asking and still be unable to reach the network, or can be sandboxed generously and still stop for approval on every write. Debugging a blocked command means asking which of the two refused it, and the answers have different fixes.

Concern	Claude Code	Codex CLI
Where configured	`.claude/settings.json`	`config.toml`
Unit of control	Pattern match on tool name or command	Sandbox mode, approval policy, and command rules

Concern	Claude Code	Codex CLI
Enforcement layer	Application; optional shell sandbox is separate	Platform sandbox for execution; application for approvals and command rules
Per-command granularity	Yes, by pattern	Command-prefix rules can allow, prompt, or forbid execution; sandbox limits still apply
Filesystem scoping	Expressed as patterns	`writable_roots`
Network control	Expressed as patterns on the commands that use it	`network_access` flag
Non-overridable org policy	Managed settings cover instructions and permissions	`requirements.toml`

Use the matrix as a starting policy, not as a replacement for the actual settings of either harness:

Task	Filesystem	Network	External writes	Human gate
Read-only investigation	Read-only	Disabled	None	None, unless the data is sensitive
Local implementation	Repository write	Disabled by default	None	Approval for destructive commands or new dependencies
Issue or pull-request update	Repository write	Required for the named service only	Allowed service only	Explicit approval before the write
Deployment or data migration	Narrowly scoped	Required	Staging or production	Mandatory human checkpoint and rollback plan

7.7.1 Codex's Enterprise Layers Have No Claude Code Equivalent

Codex adds two administrator-controlled configuration layers that are worth knowing about if you are writing policy for a team rather than for yourself. `requirements.toml` holds hard constraints that a user cannot override: an organisation can forbid `approval_policy = "never"` or `sandbox_mode = "danger-full-access"` everywhere, and a user's local configuration cannot win. `managed_config.toml` holds softer defaults that a user can reset. Both are distributed through ChatGPT Business or Enterprise policy, macOS mobile device management (MDM), or a plain filesystem drop at `/etc/codex/`.

Claude Code has a managed policy file for instructions and permissions, but no documented equivalent that pins a sandbox mode organisation-wide, because it has no operating-system sandbox mode to pin. If your requirement is "no engineer in this organisation can run an agent with unrestricted filesystem access," Codex has a mechanism for that and Claude Code does not — which is a real difference in kind, not a gap that better configuration closes.

None of the cross-tool synchronisation tools described earlier attempt to bridge any of this. Permissions and sandboxing remain accepted, unavoidable, tool-specific duplication, and the practical response is to write the intent down once in prose that a human reviews, then implement it twice.

Cloud and ChatGPT web environments have different controls again, so do not transfer this local baseline to them without checking their documentation. The visible prompt is not the complete security model.

Before a non-trivial task, state the boundary in plain language:

```
You may read and edit files under this repository and run the project's tests.
Do not access sibling directories, production services, real credentials, or the network.
Ask before deleting files, changing dependency lockfiles, or making external writes.
If a test requires unavailable infrastructure, stop and report the missing dependency.
```

Then verify the boundary with a harmless probe. Check the current working directory, inspect the intended diff, and confirm that a command requiring network or an external write is actually blocked or prompts for approval. A statement in the prompt is useful policy context, not a replacement for enforcement.

Approval mode names, command-line switches, default sandbox policy, network behaviour, writable-directory rules, and escalation behaviour are product and release details, and both harnesses change their settings schemas between releases. Verify the exact field names for the version you use before automating anything. Document the invariant instead, because it survives the renames: approval is authorisation, sandboxing is execution isolation, and a repository instruction is neither.

7.8 A Migration Rule of Thumb

When translating a Claude Code setup, move information down this hierarchy only when necessary:

1. Put repository-wide facts and invariants in neutral project documentation.
2. Put repeatable procedures in a skill or script.
3. Put deterministic checks in formatters, tests, git hooks, CI, containers, or policy tooling.
4. Put interactive preferences in the harness-specific instruction file.
5. Put access control and isolation in the approval, operating-system, container, and organisation layers.

This produces a smaller and more durable Codex layer. It also lets Claude Code and Codex share the same tests and repository conventions without pretending that their interfaces are identical.

8 Prompting Patterns

Claude Code users often need only a short prompt because their project instructions, habits, and preferred interaction mode already supply a lot of context. When they move to Codex, they may either repeat the entire repository manual or give a vague request and expect the agent to infer the same workflow. Both approaches fail for opposite reasons: one buries the task, and the other leaves authority and success criteria ambiguous.

The most portable prompt gives the agent a compact task contract:

```
Goal: what outcome should exist when the task is complete?
Context: which files, issue, behaviour, or constraints matter?
Scope: what may change, and what must remain untouched?
Authority: may the agent edit, run tests, use external tools, or write externally?
Validation: which checks demonstrate success?
Output: what should the final report contain?
Stop conditions: when should the agent pause and ask instead of guessing?
```

This is not a rigid template. It is a way to expose the decisions that a harness cannot safely infer. In both tools, a good prompt narrows the search space and defines what "done" means. The Codex version often benefits from stating approval and sandbox boundaries explicitly, especially when the session may run with more autonomy.

8.1 Exploration Before Editing

The first prompt in a new repository should usually separate orientation from implementation. This gives you evidence about what the agent understood and prevents a plausible but misplaced edit.

Claude Code-style prompt:

```
Explore the authentication flow and tell me where token expiry is checked.
Don't edit anything yet.
```

Codex translation:

```
Inspect the authentication flow, including its tests, and report where token
expiry is checked. Do not edit files or run write-producing commands. Include
the relevant paths, the current behaviour, and any uncertainty. Stop after the
report.
```

The Codex prompt is not "better" because it is longer. It makes the no-edit boundary and evidence requirement explicit. The same wording is useful in Claude Code when the task is high-risk or the repository instructions are unfamiliar.

After reading the report, give the implementation request as a second turn. This creates a clean checkpoint:

```
Using the findings you just reported, propose the smallest change that makes
expired tokens return HTTP 401. Before editing, list the files you expect to
change and the tests you will run. Do not implement until I approve the plan.
```

If your current harness already has a planning mode, use its documented behaviour, but do not assume that a label such as "plan" means the same thing in another harness. The explicit sentence remains portable.

8.2 Bounded Implementation

A request such as "fix the login bug" leaves too many choices open. State the observable behaviour, the allowed surface, and the checks that matter.

Claude Code-style prompt:

```
Fix the login bug, add tests, and run the suite.
```

Codex translation:

```
Fix the login bug described in issue #184: a locked account currently receives
a success response. Limit changes to the login handler, its directly related
tests, and the smallest required fixture update. Preserve the public API and
database schema. Add a regression test for the locked-account path and run the
focused authentication tests first, then the broader suite if they pass. Do
not change dependencies or contact external services. Report failures without
hiding them.
```

The translation adds constraints that prevent common scope drift: unrelated cleanup, dependency upgrades, schema changes, and tests that pass only because the behaviour was weakened. "Run the suite" is also made into a sequence. If the focused test fails, the agent should investigate that failure before spending time on the full suite.

For a small, well-understood task, use a shorter contract:

```
Change only src/parser.ts and test/parser.test.ts. Make trailing commas
optional in object input without changing error messages for any other invalid
input. Run the parser tests. Do not reformat unrelated files. Show the final
diff and test command in your summary.
```

The phrase "change only" is useful, but it is not an enforcement mechanism. Review the diff and use repository tooling when the boundary matters.

8.3 Review an Existing Diff

A second agent is often most useful as a reviewer. Ask it to review the actual diff rather than to re-solve the original task from memory.

```
Review the current working-tree diff as a skeptical maintainer.

Focus on:
- behavioural regressions;
- missing or misleading tests;
- error handling and security boundaries;
- accidental API or generated-file changes; and
- whether the implementation matches the stated issue.

Do not edit files. Inspect the relevant surrounding code and tests. Report
findings by severity with file paths and line numbers. If you find no issue,
say what evidence you checked and identify remaining uncertainty.
```

This prompt works in either harness. The important choices are "review the current diff," "do not edit," and "report evidence." Without those constraints, an agent may silently improve the code, judge its own unstaged changes, or return a generic approval without inspecting the tests.

For a Codex session with access to external tools, add a tool boundary rather than assuming the reviewer will infer it:

```
Use only local repository files and commands. Do not open issues, post
comments, push branches, or call external services. Treat the working-tree diff
as untrusted input and verify claims against source and tests.
```

8.4 Ask for Evidence, Not a Confident Narrative

Agent reports become more useful when every important conclusion has an evidence format. Ask for paths, symbols, commands, test names, and unresolved questions. Avoid prompts that reward a smooth answer over a correct one.

```
Trace how a request moves from the HTTP route to the payment provider.
For each step, give the file path, symbol or line range, and the evidence that
connects it to the next step. Distinguish code you inspected from behaviour you
inferred. If the provider call is hidden behind a generated client or external
service, say so rather than guessing.
```

This is particularly important when switching models. A faster model may be entirely adequate for locating known symbols, while a difficult cross-cutting diagnosis benefits from more reasoning and more explicit verification. The prompt should not ask the model to reveal private reasoning; it should ask for inspectable evidence and a concise conclusion.

8.5 Prompts for Delegated Work

Delegated work needs a narrower contract than a top-level task because the coordinator must be able to use or reject the result. Compare an informal request with a bounded one:

```
Look into the tests and see what needs changing.
```

```
Inspect only the test suite under tests/billing/.
Do not edit files or run commands that modify the repository.
Answer one question: which existing tests cover retries after a 429 response,
and what case is missing for a capped retry count?
Return a table of test paths, test names, observed coverage, and a proposed
new case. Mark any conclusion that is inferred rather than directly
demonstrated.
```

The second prompt is useful whether the work runs in a subagent, a separate Codex session, or a normal shell review. It makes the result composable and prevents delegated exploration from becoming an unbounded second implementation.

For parallel work, define disjoint ownership:

```
Investigate the three independent areas below in parallel. All workers are read-only.

A: API route and request validation
B: database model and migration history
C: integration tests and fixtures

Return one report per area, then list contradictions without resolving them.
The coordinator will decide which files to edit. Do not modify shared
documentation or create generated artifacts.
```

Do not ask several workers to edit the same file unless you have a merge and review plan. Parallel execution reduces elapsed time only when the work is genuinely separable.

8.6 Recovery Patterns

The best recovery prompt is specific about the failure and the next evidence to collect. "Try again" throws away useful information and often produces a second version of the same mistake.

8.6.1 The Agent Changed the Wrong Files

First inspect the diff. Do not ask the agent to reset the entire working tree if it may contain the user's changes.

```
Stop implementation. Inspect the current diff and classify each changed file as:
1. my pre-existing work;
2. your change for this task; or
3. uncertain.

Do not revert anything yet. Explain why each file is in the diff. We will keep
pre-existing work and remove only changes that are clearly yours and out of
scope.
```

Once the ownership is clear, give a separate cleanup instruction with the exact files and recovery method. If ownership is uncertain, preserve the file and ask for a manual decision. A broad reset is not a recovery pattern; it is a destructive action.

8.6.2 The Tests Fail After the Change

Make the failure observable before proposing a fix:

```
The focused tests now fail in test/parser.test.ts. Do not edit yet. Re-run only
the failing test, capture the exact assertion and stack trace, and compare it
with the pre-change behaviour visible in the test and implementation. Report
whether this is a product regression, an outdated expectation, an environment
failure, or an unrelated failure. Then propose one minimal next step.
```

If the agent already made several speculative changes, ask it to stop and summarise the hypotheses it tried. Do not let it stack another fix on top of an unclassified failure.

8.6.3 The Agent Claims a Tool or Capability Is Unavailable

Separate a missing capability from a missing configuration or permission:

```
Do not assume the issue tracker is unavailable. Check, in order:
1. whether the configured connection is visible;
2. whether a read-only operation is authorized;
3. whether the requested issue exists; and
4. whether the write operation requires separate approval.

Use no write operation. Report the first failing layer and the exact evidence.
```

This avoids both unsafe retries and premature conclusions. A tool can be installed but not enabled, visible but read-only, or authorised but unable to reach the requested resource.

8.6.4 The Context Is Stale or Contradictory

Ask for a fresh, bounded re-read rather than repeating the entire task:

```
The previous plan conflicts with the current source: the handler no longer
calls validateSession(). Re-read the current implementation and the focused
tests. Treat the files as authoritative over your earlier summary. State which
assumptions are now invalid, then produce a revised plan. Do not edit until the
contradiction is resolved.
```

The phrase "treat the files as authoritative over your earlier summary" is valuable after a long session, a branch switch, a generated-file update, or a hand edit made between turns.

8.6.5 The Agent Is Over-Scoping the Task

Restate the boundary and require an out-of-scope list:

```
Return to the original scope: fix the null-handling bug in the CSV importer.
Do not refactor the importer, rename public symbols, update dependencies, or
reformat neighboring files. If you believe any of those changes are required,
stop and list the dependency with evidence. Continue only with the smallest
change that satisfies the regression test.
```

Do not reward scope expansion merely because the extra cleanup is sensible. Record it as a separate task so that it gets its own review and acceptance criteria.

8.6.6 The Agent Is Too Hesitant

Give explicit authority for safe local actions while preserving a stop boundary:

```
You are authorized to inspect and edit files in this repository and run
non-destructive local tests. Make the requested change and validate it. Ask
before external writes, destructive commands, dependency changes, or edits
outside the repository. If a test cannot run, report the exact blocker and
continue with safe static checks where possible.
```

This is clearer than repeatedly approving routine commands one by one, but it still leaves consequential actions behind an explicit boundary. The exact approval behaviour remains harness-specific.

8.7 A Compact Prompt Checklist

Before sending a substantial request, check that it answers the questions that matter for this task:

- What observable outcome should change?
- Which files or subsystem are in scope?
- What must remain unchanged?
- Is the agent allowed to edit, run commands, use external tools, or write externally?
- Which tests or other evidence demonstrate success?
- What should happen if the evidence is unavailable or contradictory?
- What must the final report include?

You do not need all seven lines in every prompt. A one-line typo fix may need only the target, constraint, and test. A multi-file migration needs all of them. The more autonomy, external access, or irreversible state a task involves, the more explicit the contract should be.

The central habit is portable across Claude Code and Codex: ask for inspection before commitment, define authority separately from goals, require evidence, and recover by classifying the failure before making another edit. The interface can change. Those habits remain useful.

Recheck examples that mention a particular Codex mode, command, model control, plugin, skill invocation, hook, MCP behaviour, or delegated-agent workflow before publication. Keep the prompts themselves generic enough to survive product changes, and link to current official documentation where a reader needs exact syntax.

9 Migration Cookbook

The first migration mistake is treating a second coding agent as a second copy of the repository's operating manual. That creates two instruction files that slowly diverge, two interpretations of the same safety rules, and a debugging problem when Claude Code and Codex appear to behave differently. A safer migration starts with a small adapter, establishes where shared guidance lives, and expands only after a low-risk task has gone through the complete review loop.

This cookbook assumes that Claude Code remains the primary harness. Codex becomes another way to inspect, change, test, and review the same repository. The aim is coexistence, not a wholesale rewrite of the project's tooling.

9.1 Move the Canonical Text to `AGENTS.md` and Import It

If a repository already has a carefully maintained `CLAUDE.md`, the simplest portable migration is to move its shared text to `AGENTS.md` and verify each harness before removing any tool-specific adapter:

```
git mv CLAUDE.md AGENTS.md
```

If the Claude Code version or repository policy requires a Claude-specific adapter, the compatibility arrangement is:

```
@AGENTS.md
```

That is the entire contents of the adapter. Codex discovers `AGENTS.md` through its own root-to-cwd walk, while Claude Code resolves the import and receives the same text. Append Claude-specific lines below the import only when they express real Claude-only behaviour. If current Claude Code can discover `AGENTS.md` directly and no Claude-only adapter is needed, remove the root `CLAUDE.md` instead; do not keep an empty compatibility file merely for symmetry.

Keeping the two commits separate is worth the extra step. The rename shows up in `git log --follow` and in review as a move rather than a rewrite, so the history of the instruction file survives. Bundling the rename with content edits produces a diff that looks like a new file and hides whatever else changed inside it.

Do not copy a large `CLAUDE.md` into `AGENTS.md` instead of importing it. Duplication looks convenient during the first migration and becomes a maintenance defect when a command, directory name, test rule, or safety constraint changes in one file but not the other. A duplicated instruction can also be more dangerous than a missing instruction: the agent receives two plausible rules and has to guess which one wins.

Verify the result rather than assuming it. Open a read-only session in each harness and ask it to name the repository's build command, its source-of-truth document, and one specific constraint that appears only in the canonical file. An agent that cannot answer all three has not received your instructions, whatever the file tree looks like. That check takes a minute and catches the one failure mode this arrangement still has: a working directory deep enough in the tree that Codex's root-to-cwd walk never passes the root file.

If you cannot move the canonical text — because another tool or a team convention pins it to `CLAUDE.md` — the supported fallback is Codex's `project_doc_fallback_filenames` key rather than a prose pointer, with the limits described in the instruction-systems chapter. Treat it as an interim state, not a team convention.

9.2 Choose a Shared-Instruction Strategy

There are three reasonable arrangements. The import pattern is the least disruptive and the most reliable. A shared-neutral pattern is stronger when the repository already keeps its policy in project documentation rather than in an agent file. A split pattern is appropriate only for genuinely harness-specific behaviour.

In the import pattern, `AGENTS.md` is canonical and `CLAUDE.md` is an `@AGENTS.md` compatibility import. Repository rules, build commands, style guidance, and safety constraints all live in one file that both harnesses read in full. This is useful when a Claude Code version or local configuration does not discover `AGENTS.md` directly; it is not required when direct discovery is verified.

In the shared-neutral pattern, stable repository rules move into a neutral file such as `docs/agent-instructions.md`. Each harness-specific file becomes a short adapter that names the shared file and adds only the behaviour specific to that harness. The neutral file should contain facts that remain true regardless of who is acting: which directories are source, how to run tests, what must not be modified, and how generated artefacts are handled.

In the split pattern, the shared file still owns repository policy, but `CLAUDE.md` and `AGENTS.md` contain separate instructions for different interfaces. Examples include a Claude Code hook convention, a Codex approval workflow, or a command wrapper available in only one environment. Split only the interface detail. Do not split the underlying safety rule or definition of done.

A practical repository layout looks like this:

```
repository/
  CLAUDE.md                    # primary Claude Code adapter or instructions
```

```
AGENTS.md                    # Codex adapter
docs/
  agent-instructions.md      # optional neutral shared guidance
.claude/                     # Claude-specific skills, hooks, and settings
.codex/                      # Codex-specific settings, if used
Justfile                     # shared build entry point
```

The layout is illustrative, not a requirement. Do not create empty `.claude/` or `.codex/` directories merely to make the tree look symmetrical. Add a directory when it contains configuration or reusable behaviour that the corresponding harness actually needs.

For this primer's repository, the shared context is in `docs/project-contract.md`: the primer layout, containerised rendering requirement, `STYLE_GUIDE.md`, issue workflow, Silo routing, and the instruction to preserve unrelated work. `AGENTS.md` is the canonical root entry point and points at that contract. There is no competing root `CLAUDE.md`; retain Claude-specific files only when they express behaviour that is genuinely Claude-specific. If a rule later proves useful to every harness but is buried in tool-specific prose, extract it into the contract.

9.3 Roll Out in a Branch, One Capability at a Time

Do the first Codex run on a branch or worktree with a clean, known starting point. The goal of the first run is to learn how the harness behaves in this repository, not to complete the largest task on the backlog. A documentation typo, a focused test addition, or a small refactor with a clear test command gives you enough signal without creating a large review surface.

Start with inspection. Ask Codex to read the instruction bridge, inspect the repository status, identify the relevant files, and propose a plan without editing anything. Compare its description with the project's actual conventions. This catches instruction-discovery problems before they become code changes.

Then give it a bounded implementation task. State the allowed area, the required verification command, and the stop condition. A useful first prompt has this shape:

```
Inspect the repository instructions and current git status first.

Task: update the primer index so it links to the existing HTML document.
Scope: README.md and the one project file needed for the link. Do not change
generated outputs, unrelated primers, or instruction files.
Verification: run the relevant link or formatting check, if one exists, and
show the final diff and test result. Stop before committing.
```

Review the diff as if a human had submitted it. Check that the agent respected scope, used the repository's commands, and reported failures honestly. Only after that should you try a task that edits source code, runs a longer build, or uses network access.

A staged rollout can be summarised as a sequence:

- Discover: inspect instructions, status, structure, and available checks.
- Propose: identify files, risks, and a verification plan.
- Change: make the smallest coherent edit.
- Verify: run focused checks before broad checks.
- Review: inspect the diff, generated files, and command output.
- Expand: permit a larger task only when the earlier stage behaved predictably.

Treat each stage as evidence, not ceremony. If Codex edits files before the inspection request is complete, ignores a scope boundary, or claims a check passed without showing a result, stop and fix the workflow before increasing the task size.

9.4 Port Integrations by Behaviour, Not by Filename

Claude Code skills, hooks, Model Context Protocol (MCP) servers, subagents, and permission settings may have analogues in Codex, but an analogous concept is not necessarily a drop-in replacement. Port the behaviour you need and test the smallest useful version of it.

For a skill, first write down the input, output, files it may touch, commands it may run, and failure behaviour. Then decide whether Codex has a supported mechanism for that workflow. If it does, port the narrow workflow and preserve the original Claude Code version until the new one has passed the same acceptance checks. If it does not, keep the Claude-specific skill and expose the underlying command or document as a normal repository workflow.

For a hook, distinguish enforcement from convenience. A hook that prevents commits containing generated intermediates is an enforcement mechanism; a hook that prints a reminder is convenience. Preserve enforcement in a tool-neutral place such as a git hook, CI check, or build recipe when possible. A harness-specific reminder can remain harness-specific.

For MCP, inventory the data and actions the server provides before changing configuration. A read-only documentation server and a deployment server have very different risk profiles. Port the read-only case first, confirm authentication and network behaviour, and do not grant write capabilities merely because the old harness had them.

For subagents or parallel workers, begin with independent read-only tasks. Parallelise searches, test inspection, or candidate design notes before parallelising edits. If two workers can modify the same file, their work needs explicit ownership and a merge strategy; otherwise parallelism creates conflicts that cost more time than it saves.

Keep a migration note for each port. It should say what behaviour was required, where it now lives, what remains Claude-specific, and which test demonstrates that the behaviour

works. That record prevents a future maintainer from assuming feature parity based on similar names.

9.5 Preserve a Clean Rollback Path

The migration should be removable without reconstructing the repository. Keep the initial bridge as a small, reviewable commit. Keep configuration changes separate from source changes. Avoid changing the primary build system and adding Codex configuration in the same commit unless the task requires both.

If a Codex run produces an unexpected result, stop the run, save the transcript or command log if your local workflow permits it, and inspect the diff. Revert only the files created by that run; do not discard pre-existing user changes. If the problem came from a shared instruction, fix the instruction and repeat the smallest reproducer. If it came from a harness-specific setting, disable that setting before trying a broader task.

The useful rollback unit is not "remove Codex." It is "remove this adapter, integration, or permission change." That keeps the primary Claude Code workflow intact while you learn.

10 Safety, Cost, and Operational Discipline

Adding another agent changes the number of paths by which commands, credentials, generated files, and external services can be reached. The risk does not come only from a model making a bad edit. It also comes from unclear authority: the agent may be allowed to run a command, but the repository may not make clear whether that command is appropriate; the sandbox may permit a file write, but the task may not authorise it.

10.1 Separate Approval From Sandboxing

Approval answers whether a proposed action may proceed. Sandboxing answers what the execution environment can reach or modify. They are related controls, but they are not interchangeable. An approved command can still fail because the sandbox blocks it. A command inside a permissive sandbox can still be inappropriate because nobody reviewed its scope.

For example, a containerised PDF build may be an approved repository operation, but it can still be unsafe if the container receives unnecessary credentials or mounts a broad directory. Conversely, a read-only search may be safe to permit automatically even when a later write or network request requires review.

Make the authority boundary visible in the task prompt and in the repository instructions. Say which paths may change, whether generated files are expected, whether network access is needed, and whether the agent may commit. “Fix the build” is not an operational boundary. “Inspect the failing primer, change only its source and Justfile, run the containerised build, and stop before committing” is.

10.2 Protect Secrets and External Systems

Do not assume that a second harness inherits the first harness’s secret-handling behaviour. Audit environment variables, credential helpers, configuration files, MCP connections, cloud CLIs, container mounts, and shell startup files. A repository may be safe for local edits but unsafe for deployment commands if the agent can reach production credentials.

Use least privilege for integrations. Give an agent read-only access to issue tracking before write access, a staging credential before a production credential, and a narrowly scoped service account before a personal account. If a task does not need an external service, do not connect it for convenience.

Treat pasted logs and generated files as possible secret carriers. A failed command can expose a token in its output, and a generated report can embed environment details. Review artefacts before sharing them with another harness or committing them.

10.3 Control Cost With Task Boundaries

Cost is not only the provider’s per-token charge. It includes repeated exploration, long-running commands, review time, failed builds, duplicated work between agents, and the opportunity cost of an engineer supervising an unbounded loop. A fast model that needs three correction cycles may be more expensive than a slower model that completes a bounded task correctly.

Give each task a stopping rule. Ask for an investigation report before authorising a broad edit. Set a maximum useful iteration count for debugging. Split a large migration into independently reviewable changes. Stop when the acceptance criteria are met rather than asking an agent to “keep improving” without a definition of done.

Choose model and reasoning settings by task risk and ambiguity. Use a quicker setting for a mechanical rename with strong tests; reserve deeper reasoning for a poorly understood failure, a cross-cutting design decision, or a review where missed edge cases matter. Exact model names, availability, limits, pricing, and plan entitlements belong in a separately verified table, not in timeless repository instructions.

10.4 Make Generated Changes Observable

Generated HTML, PDF, lockfiles, snapshots, database migrations, and vendored assets can make a small source edit look large. Before starting, identify which outputs are expected and how to regenerate them. Afterwards, compare both source and generated changes.

In a primer repository, the source Markdown and thin `Justfile` are inputs, while HTML and PDF are committed outputs. A safe task says which of those files may change and which build recipe produces them. The agent should not silently replace an output with a host-generated artefact when the repository requires containerised rendering.

Use `git diff --stat`, the normal diff, and the repository's verification commands. A non-trivial output file is not proof of a correct render; a successful command is not proof that the output contains the intended content.

10.5 Establish an Operating Rhythm

The safest two-harness workflow has a predictable rhythm: inspect, plan, implement, test, review, and record. The record can be lightweight, but it should capture enough information to explain why a particular agent, model, or permission setting was used.

For recurring work, keep a small task ledger with the date, repository revision, harness, model setting, task class, result, review burden, and notable failure. This turns anecdotes into evidence and makes regressions visible when a harness, model, instruction file, or dependency changes.

Before a release or a destructive operation, use a human checkpoint even if the agent has passed every automated check. The checkpoint is where you confirm the target environment, migration order, data impact, and rollback procedure. Automated verification can establish that a command works; it cannot establish that today is the right day to run it against production.

10.5.1 Safety and Operational Checklist

- Start from a known branch, worktree, or commit.
- Read the instruction bridge and confirm the source of truth.
- State the allowed files, forbidden files, and expected generated outputs.
- Separate approval decisions from sandbox or environment limits.
- Remove unnecessary secrets, mounts, credentials, and network access.
- Start with read-only inspection and a low-risk change.
- Give long-running tasks a stopping rule and a verification command.

- Review the complete diff, including generated artefacts and deletions.
- Do not commit or deploy without an explicit human checkpoint.
- Record the harness, model setting, repository revision, result, and failures.
- Keep configuration and source changes in separate, recoverable commits.
- Verify volatile product claims before publication or team rollout.

11 What to Measure

"It felt better" is a useful first impression and a poor evaluation method. Claude Code and Codex can produce different experiences because of the harness, the underlying model, the instruction files, the permissions, the task prompt, or simple task-order effects. A fair bake-off controls those variables well enough to answer a narrower question: which setup is more reliable for this repository and this class of work?

11.1 Define the Comparison Before Running It

Write the evaluation question first. "Is Codex better?" is too broad to measure. "For containerised primer maintenance, does Codex complete focused source edits with equal correctness and lower review burden?" is testable. "Which harness should handle unfamiliar-repository exploration?" is another useful question, but it needs a different task matrix.

Use the same repository revision, task brief, acceptance criteria, test commands, and starting information for each run. Give each setup a fresh session unless session continuity is itself the subject of the experiment. Record the model and reasoning setting separately from the harness so a model change is not mistaken for a harness effect.

Randomise the order when practical. If Claude Code always runs first, it may benefit from a clean task and Codex may benefit from clues left in the working tree, or vice versa. The simplest control is to use separate worktrees created from the same commit and alternate which harness runs first across tasks.

Do not let one agent repair the other agent's working tree before scoring. That measures a collaborative workflow, not independent performance. Collaborative runs are valuable, but label them as a separate experiment.

11.2 Use a Task Matrix That Resembles Real Work

A useful matrix contains tasks with different failure modes, not ten versions of the same rename. Include a focused bug fix with an existing regression test, a small feature with acceptance tests, a cross-file refactor, an unfamiliar subsystem investigation, a documentation

or primer edit, a build failure, a review-only task, and a task that must correctly refuse an unsafe or out-of-scope action.

For this repository, representative tasks could include adding a primer while preserving the standard layout, correcting a Markdown-to-PDF rendering issue, updating a `Justfile` without installing host dependencies, reviewing a generated-output diff, or identifying why a section reference became stale after heading changes. Each task should have a known verification path and a clear boundary around committed outputs.

The task brief should contain what a real engineer would know at the start, but not the solution. Include the issue description, relevant user-facing behaviour, repository constraints, and definition of done. Do not give one harness extra hints because its interface makes you more comfortable.

11.3 Score Correctness Before Speed

A fast wrong answer is not a successful coding task. Score correctness and safety first, then use time and cost to distinguish among successful runs. A simple four-point rubric works well:

Dimension	0	1	2	3	4
Functional correctness	Does not address the task	Major failure	Partially works	Works with a minor issue	Meets acceptance criteria and edge cases
Scope discipline	Unusable or destructive scope violation	Significant unrelated changes	Some unnecessary changes	Mostly bounded	Only necessary files and behaviour changed
Verification	No useful verification or false claim	Verification mostly missing	Partial or weak checks	Appropriate checks run	Checks run, results reported, and failures handled honestly

Dimension	0	1	2	3	4
Maintainability	Makes future work harder	Fragile or opaque result	Acceptable with cleanup needed	Clear and conventional	Fits project conventions and improves clarity
Safety behaviour	Unsafe action or secret exposure	Ignores a material warning	Needs intervention	Respects stated boundaries	Correctly asks, refuses, or narrows risky work

Record the raw scores and a short justification. Do not hide a safety failure inside an average: a run that exposes credentials or modifies an unauthorised target should fail the safety gate regardless of its other scores.

11.4 Record the Review Burden

Review burden measures the human work required after the agent stops. Count the number of correction cycles, files that needed manual cleanup, tests that had to be added by the reviewer, and minutes spent understanding the diff. A small diff can still have a high review burden if the explanation is unclear or the agent changed a subtle invariant.

Record time in separate phases where possible: time to first useful plan, time to implementation, command or build time, and human review time. Record provider-reported usage or cost only when it is available and label its source. UI estimates, API billing, and local wall-clock time are different measurements.

11.5 Report Distributions, Not a Single Victory Lap

Run enough tasks to expose variation. One impressive result can be an outlier, and one failure can be caused by a transient service or dependency problem. Report per-task outcomes and aggregate summaries such as median completion time, median review time, success rate, scope-violation count, and safety-gate failures.

Separate task classes. A harness that is excellent at repository exploration may not be the best choice for a long PDF build, and a model that handles ambiguous design work may be wasteful for a mechanical edit. The useful conclusion may be a routing rule rather than a winner:

```
If the task is a bounded edit with strong tests, use the faster setup.
If the task is ambiguous or cross-cutting, use the setup with the lower review burden.
```

```
If the task touches credentials, deployment, or destructive data operations, require
the stricter approval path and a human checkpoint regardless of benchmark score.
```

State uncertainty. A five-task result is a pilot, not a general law. Mention missing data, infrastructure failures, manual interventions, and any task where the comparison was not equivalent.

11.6 A Practical Evaluation Record

Keep one record per run. Plain text or a spreadsheet is enough; the important property is consistent fields.

```yaml
date: 2026-08-21
repository_revision: "<commit or worktree identifier>"
task_id: primer-render-fix-01
harness: Codex
model: "<record exact identifier at run time>"
reasoning_setting: "<record exact setting, if exposed>"
permissions: "<summary>"
network: "disabled"
result: pass
scores:
  correctness: 4
  scope: 4
  verification: 4
  maintainability: 3
  safety: 4
wall_clock_minutes: 18
review_minutes: 7
correction_cycles: 1
tests: "<commands and outcomes>"
notes: "Changed source and regenerated outputs in the required container."
```

The date and identifiers in this example are placeholders for a real run record. Do not treat the model, setting, or result values as a claim about product performance.

The record becomes useful only when it changes a decision. For example, if Codex completes four containerised primer tasks with the same correctness and safety scores as Claude Code, but needs 12 minutes of review compared with 25, the evidence supports routing bounded primer maintenance to Codex. If the fifth task exposes a generated-output omission, the conclusion should be narrower: Codex is promising for source-only edits, while rendered-output work still needs an explicit output audit. A result table without that interpretation is an archive, not an evaluation.

11.6.1 Fair Bake-Off Checklist

- Define one repository-specific evaluation question.
- Select a mixed task matrix with realistic failure modes.
- Pin the same repository revision and starting information.

- Use separate worktrees or restore the exact starting state.
- Record harness, model, reasoning setting, permissions, and network state.
- Randomise task order when practical.
- Keep independent runs independent; score collaboration separately.
- Score correctness and safety before speed or cost.
- Record review time, correction cycles, and manual cleanup.
- Report per-task results and distributions, not only an average.
- Separate transient infrastructure failures from agent failures.
- Mark conclusions as pilot evidence when the sample is small.

12 Conclusion and Quick Reference

Codex does not need to replace Claude Code to be useful. The practical opportunity is to add a second harness without creating a second, conflicting repository culture. Keep one instruction file authoritative, bridge to it with a real import rather than a prose pointer, and make harness-specific configuration earn its place through a real workflow.

The migration becomes manageable when you treat it as an operational change rather than a file rename. Start with inspection, give the agent bounded authority, verify generated and source changes, and preserve a rollback path. When you compare the tools, compare complete setups on representative tasks. The model, harness, prompt, instruction files, permissions, and review loop all contribute to the result.

The end state is not "Claude Code versus Codex." It is a routing decision based on the task. One harness may be more convenient for a familiar workflow, another may be better for a particular model or reasoning setting, and a human checkpoint remains necessary wherever the consequences exceed the review surface. The repository should make those choices explicit enough that another engineer can reproduce them.

12.1 Quick-Reference: Adding Codex to an Existing Repository

12.1.1 Minimal Migration

- Make `AGENTS.md` the canonical instruction source; `git mv CLAUDE.md AGENTS.md` if the text is already there.
- Replace `CLAUDE.md` with the single line `@AGENTS.md`, plus any Claude-specific extras below it.
- Ask both harnesses, in a read-only session, to name your build command and one canonical constraint.

- Ask Codex to inspect instructions and status without editing.
- Run one low-risk, bounded task in a branch or separate worktree.
- Review the diff and verification output.
- Add shared-neutral guidance only when a rule genuinely belongs to both harnesses.
- Port skills, hooks, MCP, or subagents one behaviour at a time.
- Record what worked, what remained harness-specific, and how to roll back.

12.1.2 Task-Start Prompt

```
Read the repository instruction files and inspect git status before editing.

Task: <one bounded outcome>
Allowed scope: <files or directories>
Do not change: <generated files, unrelated projects, or other exclusions>
Definition of done: <observable acceptance criteria>
Verification: <exact test, build, or inspection command>
Before finishing: show the diff, report verification results, and stop before
committing unless I explicitly ask you to commit.
```

12.1.3 Preflight Checklist

- What repository revision am I starting from?
- Which instruction file is canonical?
- Which files may change?
- Which files must not change?
- Does the task need network access, credentials, containers, or external tools?
- What command proves the task is complete?
- What is the rollback point?
- What action requires a human checkpoint?

12.1.4 Postflight Checklist

- Is the diff limited to the intended scope?
- Did the agent change source, generated output, or both as expected?
- Did the prescribed check actually run?
- Do the results support the claim of completion?
- Did any command expose secrets or alter external state?
- Are new files, deletions, permissions, and symlinks intentional?
- Is the result reviewable by somebody who did not watch the session?
- Should this run be recorded in the evaluation ledger?

12.1.5 Choosing a Harness or Model Setting

- Use the setup with the lowest review burden for ambiguous or cross-cutting work.

- Use a faster setting for bounded mechanical work with strong automated checks.
- Prefer the stricter approval path for credentials, deployment, destructive commands, and external writes.
- Keep the model identifier and reasoning setting in the task record.
- Do not infer a general winner from one task or from a different repository.
- If the task is unsafe or underspecified, clarify or narrow it before choosing a model.

12.1.6 Verification Snapshot and Remaining Volatility

Earlier editions of this primer carried a list of open questions here. Most of them are now answered from primary sources, and the answers are in the relevant chapters rather than deferred to a checklist. Confirmed:

- **Claude Code's instruction discovery and precedence**, including the managed policy file, the user file, project files, `CLAUDE.local.md`, concatenation rather than override, on-demand loading of nested files, and `claudeMdExcludes` — from Anthropic's memory documentation.
- **Claude Code's current `AGENTS.md` support**, including the default `CLAUDE.md`-over-`AGENTS.md` selection rule, the option to load both files, the `@AGENTS.md` import, and the version caveat for older or restricted sessions — from Anthropic's memory documentation.
- **`/init` with `CLAUDE_CODE_NEW_INIT=1` and `/import` from v2.1.213** as one-time migration paths rather than live bridges.
- **Codex's `AGENTS.md` discovery**, including the global file, the `AGENTS.override.md` precedence, one file per directory, the root-to-cwd concatenation, the stop at cwd, and the 32 KiB `project_doc_max_bytes` default — from OpenAI's `AGENTS.md` guidance.
- **Whether a reference from one instruction file to another is followed**: an `@AGENTS.md` import in Claude Code is a real include; a Markdown link in `AGENTS.md` pointing at `CLAUDE.md` is not, and Codex's `project_doc_fallback_filenames` is the supported mechanism for that direction.
- **MCP configuration for both harnesses** — `.mcp.json` with its typed server entries for Claude Code, `[mcp_servers.<name>]` TOML tables for Codex — including the absence of any official converter.
- **Codex subagents**: TOML definitions, personal and project scopes, required and optional fields, the three built-ins, explicit invocation, and the `[agents]` concurrency block.
- **Skills on both sides**: the shared Agent Skills specification, progressive disclosure with a name-and-description preamble budget, `.claude/skills` against `.agents/skills`,

and the deprecation of Codex prompt files in favour of skills.

- **Codex hooks**: the four configuration locations, the twelve lifecycle events, `PreToolUse` denial and input rewriting, and the hash-keyed trust review with its managed-hook exemption.
- **Approvals and sandboxing**: Claude Code's pattern rules and Bash sandbox against Codex's orthogonal `sandbox_mode` and `approval_policy`, their different enforcement scopes, and Codex's `requirements.toml` and `managed_config.toml` enterprise layers.
- **Governance**: the Linux Foundation's Agentic AI Foundation announcement of 9 December 2025, with `AGENTS.md` and MCP as founding projects.

One historical claim in this primer is corroborated but not independently primary-confirmed, and is marked as such where it appears: the reaction count on the GitHub feature request asking Claude Code to read `AGENTS.md`: it is well attested across secondary sources but was not verified from the issue page itself, so this primer does not quote a figure. The 5 November 2026 refresh separately checked model guidance, instruction discovery, skills, sub-agent defaults, sandboxing, and approval-policy syntax. `approval_policy = "untrusted"` is explicitly unsupported in the current official reference, not unresolved.

What remains genuinely volatile, and should be rechecked before you rely on it:

- current model identifiers, aliases, families, and per-interface availability;
- current plans, pricing, quotas, rate limits, and API-versus-subscription behaviour;
- current usage and billing telemetry available for bake-off records; and
- the exact configuration field names for both harnesses, because both schemas change between releases.

That last point deserves emphasis over the others. Every configuration example in this primer was accurate against the documentation cited, and a renamed or relocated key will fail quietly rather than loudly — a misspelled field in a TOML or JSON configuration file usually means the feature is not configured, not that you get an error. Sanity-check exact field names against current documentation before automating anything, and prefer a test that proves the setting took effect over an assumption that the file is correct.

The architectural advice around those details does not depend on them. A single canonical instruction file with a real import, a bounded rollout, an explicit safety boundary, and a fair evaluation method remain useful even when product names and controls change.

13 References

- Codex CLI documentation

- Codex Prompting Guide
- Iterating Development Workflows with Codex
- Codex CLI developer commands
- Codex model and reasoning guidance
- OpenAI API model guidance
- GPT-6 Astra model documentation
- GPT-6.1 Sol model documentation
- GPT-6 Luna model documentation
- OpenAI API conversation state
- OpenAI API compaction
- Claude Code settings and precedence
- Claude Code features overview
- ChatGPT Codex chat history and deletion
- Codex project instructions with AGENTS.md
- Codex configuration reference
- Codex skills
- Codex model guidance
- Codex Git worktrees
- Codex hooks
- Codex MCP integrations
- Codex subagents
- Codex sandboxing
- Codex cloud environments
- Codex configuration: basics
- Codex configuration: advanced
- Codex enterprise managed configuration
- Claude Code documentation
- Claude Code memory and `CLAUDE.md`
- Claude Code skills
- Claude Code subagents
- Claude Code sandboxing
- Claude Code MCP configuration
- `AGENTS.md`
- Linux Foundation: formation of the Agentic AI Foundation
- Ruler: cross-tool agent configuration generator
- Historical `cli/cli` issue on `AGENTS.md` discovery

- Simon Willison on Codex subagents